Federated Continual Learning for Human-Robot Interaction

Luke Guerdan
Churchill College

UNIVERSITY OF CAMBRIDGE

A dissertation submitted to the University of Cambridge in partial fulfilment of the requirements for the degree of Master of Philosophy in Advanced Computer Science

University of Cambridge
Computer Laboratory
William Gates Building
15 JJ Thomson Avenue
Cambridge CB3 0FD
United Kingdom

Email: lg619@cl.cam.ac.uk

June 4, 2021

Declaration

I Luke Guerdan of Churchill College, being a candidate for the M.Phil in Advanced Computer Science, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.

Total word count: 14948

Signed: 

Date: 04/06/2021

Acknowledgements

I am very grateful for the support of the Affective Intelligence and Robotics Laboratory while working on my dissertation. I would especially like to thank Dr. Hatice Gunes for her guidance, and encouragement to explore new ideas while maintaining a thorough research process. Her mentorship has taught me valuable experiment design, research organization, and communication skills, which I hope to take with me to future endeavors. I would also like to thank members of the Affective Intelligence and Robotics Laboratory for their helpful feedback during lab presentations. Dr. Sinan Kalkan and Nikhil Chilimani in particular have been very helpful with answering my continual learning questions.

Abstract

Federated Continual Learning for Human-Robot Interaction

From learning assistance to companionship, social robots promise to enhance many aspects of daily life. However, social robots have not seen widespread adoption, in part because (1) they do not adapt their behavior to new users, and (2) they do not provide sufficient privacy protections. Centralized learning, whereby robots develop skills by gathering data on a server, contributes to these limitations by preventing online learning of new experiences and requiring storage of privacy-sensitive data.

In this work, we propose a decentralized learning alternative that improves the privacy and personalization of social robots. We adopt two machine learning approaches, Federated Learning and Continual Learning, to capture interaction dynamics distributed physically across robots and temporally across interactions. After formalizing long-term Human-Robot Interaction (HRI) as a Federated Continual Learning (FCL) problem, we evaluate a series of FCL approaches in different interaction scenarios. We compare performance across a set of five proposed criteria and develop a new algorithm – Elastic Transfer – which improves decentralized learning by encouraging robots to agree on the most important model information.

Results indicate that decentralized learning provides similar performance to a centralized baseline while adding personalization and privacy benefits. Our analysis also illustrates that the proposed evaluation criteria capture nuanced distributed learning trade-offs and suggests that the proposed ET approach may improve FCL performance across several performance dimensions. This work opens the door to future research examining decentralized learning for Human-Robot Interaction.

Total word count: 14948

Contents

Nomenclature

Abbreviations

AMSE Average Mean Squared Error

APD Additive Parameter Decomposition

BWT Backward Transfer

CL Continual Learning

Class-IL Class Incremental Learning

CNN Convolutional Neural Network

DGL Deep Graph Library

Domain-IL Domain Incremental Learning

EWC Elastic Weight Consolidation

FCL Federated Continual Learning

FedAvg Federated Averaging

FIM Fisher Information Matrix

FL Federated Learning

FWT Forward Transfer

GCN Graph Convolutional Network

GNN Graph Neural Network

HRI Human-Robot Interaction

MAP Maximum A Posteriori

MSE Mean Squared Error

RNN Recurrent Neural Network

SAN Socially-Aware Navigation

SGD Stochastic Gradient Descent

STL Single Task Learning

Task-IL Task Incremental Learning

Symbols

ℓ Loss function

γ Weight decay

λ EWC, FedCurv, and FedWeIT Regularization strength

$\mathcal{D}$ Supervised learning dataset

$\mathcal{N}$ Gaussian distribution

μ FedProx regularization strength

θ Neural network parameters

θ^* Optimal model derived from training

θ_G Global FL model

C Set of clients indexed by c

C_r Set of selected clients in a round

D Number of clients dropped per round

E Epochs of local training each communication round

F Fisher information matrix

H Neural network function

M External memory store used by continual learning

P Train-test performance matrix

R Communication rounds per task indexed by r

T Continual learning tasks indexed by t

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

As robot systems improve, they are moving from laboratory environments into our daily lives. The homes of tomorrow may enlist Jibo, "the world's first family robot", to lend support with cooking, family photos, and entertainment [1]. The dog-like robot MIRO might bring companionship to the aging population in tomorrow's care homes [2]. In the classroom, humanoid robots like Pepper may provide emotionally intelligent tutoring to students who need learning assistance [3]. Yet, several key barriers remain before social robots such as these can become fully integrated into daily life.

One barrier to widespread social robot adoption is *personalization*. Whereas robots of today are designed for single encounters based on pre-configured abilities, future systems require adaptation over long-term interactions. For example, longitudinal HRI research has found that robots who tailor their behavior towards users' personality, preferences, and interaction style foster more engagement and long-term acceptance [4]. A further barrier to social robot adoption is the lack of *privacy protections*. Because robots like Jibo, MIRO, and Pepper will support humans with sensitive activities such as learning and eldercare, data protection must be considered from the ground up. However, existing social robot systems do not emphasize privacy, thus reducing user trust in the system and limiting widespread adoption [5, 6].

An underlying cause of these barriers is the *centralized machine learning* (ML) process robots use to develop intelligent behaviors. Centralized ML works by (1) gathering user data in a single location such as a server and (2) training a model to perform a new function (i.e. classifying user emotions) using the full dataset. However, this process limits robots' ability to personalize to new users, because doing so would require uploading raw user data, retraining the model, and redistributing the model to all devices. Moreover, in a social robotics scenario, storing user data on a server imposes privacy vulnerabilities. Instead, by leveraging *decentralized learning*, whereby robots leave raw data on robot devices and learn by collaboratively exchanging model information, it may be possible to support personalization while also protecting privacy.

1.2 Project Overview

This work proposes a decentralized learning approach for long-term HRI. We combine two ML paradigms – Federated Learning (FL) and Continual Learning (CL) – to model interactions that are distributed *physically* across robots and *temporally* across long-term interactions. Federated learning is a paradigm that trains a model by combining knowledge from a network of devices called clients [7]. During each training round, a global model is sent to clients, refined locally with user data, and sent back to the server, where updates are combined into a new model. This eliminates the need for centralized data storage. Continual learning is a paradigm that trains a model *online* as it learns new experiences [8]. CL approaches learn new knowledge and skills (called tasks) incrementally as they try to retain knowledge from previous experiences. In this work, we adopt a recently proposed Federated Continual Learning (FCL) approach that combines FL and CL [9] to address the challenges of long-term HRI.

Though FL, CL, and FCL have been evaluated within the ML community, these approaches have not seen widespread adoption in HRI. Therefore, the first phase of this project is to formulate long-term HRI as an FCL problem. We construct two *learning scenarios* illustrating common HRI contexts: (1) many-to-many personalization between a group of robots and individuals, and (2) one-to-one personalization between a single user and robot. We outline desired performance characteristics of FCL for HRI and propose an evaluation procedure for measuring them. Next, to provide a proof of concept on an HRI problem, we identify an existing HRI dataset for an algorithmic evaluation. We leverage the SocNav1 dataset [10], which models user atti-

tudes towards socially acceptable robot navigation. We establish a *centralized learning benchmark* by adopting a Graph Neural Network (GNN) architecture proposed by the SocNav1 authors. Finally, using the framework above, we reformulate the centralized benchmark into a *decentralized* FCL learning scenario.

Next, we evaluate existing FCL approaches on the HRI scenarios established during phase one. We conduct a comprehensive evaluation of 10 algorithms, including combinations of standard FL and CL approaches. We also evaluate the state-of-the-art algorithm, FedWeIT, proposed by the FCL authors [9]. In addition to comparing performance in different personalization scenarios, we also compare alternate *learning strategies*. Specifically, we examine whether it is better to learn from distributed interactions by (1) increasing model complexity (*architectural strategies*) or (2) preventing user models from diverging from one another (*regularization strategies*). We find that the widely used FedAvg FL algorithm provides decentralized performance matching the centralized benchmark. We also find that 1) methods perform better on many-to-many than one-to-one personalization scenarios, 2) combining FL with CL enables more rapid personalization, and 3) regularization strategies outperform architectural strategies.

Finally, we leverage findings from the initial evaluation to propose a novel *Elastic Transfer* (ET) algorithm. In the first step, clients use an existing Elastic Weight Consolidation (EWC) method [11] used in CL to estimate which model parameters are most important for previous tasks. After transmitting importance estimates to other clients, each client avoids overwriting other's important parameters by adding an adaptive regularization penalty proportional to other's importance estimates. We evaluate regularization-based methods in a CL-only setting, FL-only setting, and in a combined FCL setting. Results provide further evidence that FedAvg maintains strong decentralized performance across long-term HRI scenarios. Findings also indicate ET regularization improves performance compared with existing methods. In Chapter 7, we highlight opportunities for continued improvements to ET.

1.3 Contributions

This work provides three key contributions, in which we:

1. **Formulate long-term HRI as an FCL problem.** To our knowledge, this is the first work to formulate HRI as an FCL problem. We provide a flexible extension to previous CL for HRI frameworks [12] that supports *personalization* to new individuals and groups. Our proposed evaluation procedure and criteria may serve as a starting point for future FCL for HRI research.

2. **Evaluate the trade-offs of existing FCL approaches.** We conduct a comprehensive evaluation of 10 alternate FCL algorithms. Though the initial FCL work compared many of the same methods [9], our evaluation is tailored to HRI specific learning challenges, such as dataset imbalances, small dataset sizes, and annotation disagreement. We believe insights learned from this analysis may inform nascent research in areas such as *personalized FL* and *longitudinal HRI*.

3. **Propose a regularization strategy for improving FCL performance.** We draw theoretical connections between existing FL and CL algorithms to propose a new *Elastic Transfer* approach for improving knowledge sharing among clients. In contrast to the architectural approach FedWeIT, this provides the first regularization-based strategy tailored to FCL problems.

1.4 Dissertation Structure

Chapter 2 covers the background necessary to follow this report, while Chapter 3 outlines related work. Chapter 4 formulates long-term HRI as an FCL problem. Chapter 5 evaluates existing FCL approaches. Chapter 6 develops ET regularization and conducts an extended evaluation. Chapter 7 concludes by tying the contributions of this dissertation into the challenges of long-term HRI. Additional results are included in the Appendix.

Chapter 2

Background

We begin this chapter with an overview of Human-Robot Interaction (HRI), with a focus on the challenges that arise in long-term interaction contexts. We then review the centralized learning process used in existing HRI applications before introducing two paradigms – Federated Learning (FL) and Continual Learning (CL) – that can be combined to create a decentralized learning alternative.

2.1 Human-Robot Interaction

Human-robot interaction research involves the design, implementation, and evaluation of robot systems that work alongside people [13]. Though HRI has seen applications in domains such as education [13] and healthcare [14], many current systems only consider interactions during a single encounter. This may limit the trust, fluency, and overall effectiveness of human-robot collaboration [15]. To address this limitation, *long-term HRI* research examines robots that interact with end-users over weeks or months [4].

Considering multiple interactions provides an opportunity for robots to *personalize to new contexts and users* based on their specific preferences, emotions, and behaviors [15]. These adaptation approaches improve perception of robot competence [16] and increase performance of human-robot teams [17]. Additionally, by studying *longitudinal* interactions, it is possible to gain a robust understanding of how attitudes towards robots evolve [4]. However, before long-term interactions can be leveraged in real-world settings, several challenges remain [15, 4]. These include:

- **User Adaptation:** Adapting to new users poses a non-trivial technical challenge [15]. Individuals show diverse attitudes [18], preferences [19], and physiological responses [20] to robot behaviors, which can be difficult to incorporate *offline* before interaction sessions begin. Therefore, it is necessary for robots to update their behavior *online*, as experiences unfold over time.

- **Data Availability:** Because robots require time-intensive data collection protocols, long-term HRI studies have a small sample size [4]. As a result, long-term HRI datasets tend to be small. This constraint makes it difficult to rely on offline datasets while generalizing to real-world environments. Instead, it is necessary to collect additional data from users to personalize to their behaviors as interactions unfold.
- **Privacy & Data Protection:** Robots need to ensure they keep information gathered throughout interactions private and secure. This consideration is particularly important in social robotics contexts, where the robot may be assisting with rehabilitation, emotional support, or other sensitive personal activities [15].
- **Evaluation Criteria:** A final challenge for real-world longitudinal HRI is quantifying how effectively a system adapts to user-specific behaviors [15]. Existing research has not established a uniform set of criteria for assessing the benefits of a specific system [4]. Developing a set of general evaluation criteria that apply across interaction contexts remains an outstanding problem in long-term HRI.

To illustrate how these unfold within a concrete social robotics application, the next subsection introduces the HRI sub-field of Socially-Aware Navigation (SAN).

2.1.1 Proof of Concept Domain: SAN

When robots share physical space with humans, it is important that they navigate in a safe and appropriate manner. Therefore, Socially-Aware Navigation gives robots the ability to reason about the social conventions and expectations of users as they move through space [21]. Recent work proposes data-driven approaches for learning human navigation preferences automatically [22]. These methods often use supervised machine learning (ML), which trains a model to predict user comfort levels using previous navigation actions and corresponding feedback. For example, one recent work proposes a Graph Neural Network (GNN), a specific supervised ML technique, for learning user comfort with robot navigation behaviors [23].

However, these methods rely on a static data corpus gathered *offline*. This is undesirable, as humans' space preferences (1) change over time and (2) differ by background and culture [21]. This suggests that effective social-robot navigation over long-term interactions may require updating the model *incrementally* as user feedback evolves. Therefore, recent work has proposed socially aware navigation models that *incrementally* learn new social nav-

igation preferences [12]. Instead of using a *centralized learning* approach, whereby (1) a full dataset is gathered on a server and (2) a single model is trained *offline* to generalize across contexts, these works employ *decentralized learning* to update a model with new interaction data *online* [24].

2.2 Centralized Learning

To highlight the difference between centralized and decentralized learning, we provide an overview of the *centralized model development* process. We illustrate this paradigm in the specific context of neural network training. Let the set of observations $\mathcal{D} = \{(\vec{x}_i, y_i) \mid i = 1 : N\}$ contain N labeled training examples gathered in a single location such as a server. For this supervised learning problem, each training example i consists of a set of input features $\vec{x}_i$ that map to the desired output y_i. Given this central dataset, a neural network learns a function $H(\vec{x}_i; \theta) = \hat{y}_i$ that approximates the desired output by finding an optimal set of parameters θ such that the error between $\hat{y}_i$ and y_i is minimized. The degree of similarity between $\hat{y}_i$ and y_i is encoded by a *loss function*.

The Mean Squared Error (MSE) is a commonly-used loss function when y is a continuous variable. Given an MSE loss $\ell(\theta; \vec{x}, y)$, the neural network training objective is of the form:

$$\min_{\theta} \ell(\theta; \vec{x}, y) \quad \text{where} \quad \ell(\theta; \vec{x}, y) := \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2. \tag{2.1}$$

This objective function is optimized through an iterative gradient-based approach such as stochastic gradient descent (SGD) [25].

To measure how well $H(x; \theta)$ captures performance in new contexts, $\mathcal{D}$ is often partitioned into a training, validation, and test dataset ($\mathcal{D}_{train}$, $\mathcal{D}_{valid}$, $\mathcal{D}_{test}$, respectively) [26]. In this approach, equation 2.1 is optimized with respect to $\mathcal{D}_{train}$. The validation dataset $\mathcal{D}_{valid}$ is used during hyperparameter tuning and model architecture selection. Finally, $\mathcal{D}_{test}$ is held out until the final evaluation to provide an estimate of generalization capacity. Under this *centralized model development* process, after estimating generalization using $\mathcal{D}_{test}$, the model is deployed to a real-life scenario such as the SAN context outlined in Section 2.1.1. At this point, the model becomes fixed, and is not updated with new examples from $\mathcal{D}_{train}$. In practice, centralized model development with neural networks involves deep learning.

2.2.1 Deep Learning

Neural networks are organized into stacked layers. Each layer forms a functional unit such as a fully-connected [27], convolutional [28], or recurrent layer [29]. These layers may be combined to form architectures such as a Convolutional Neural Network (CNN), or Recurrent Neural Network (RNN). The process of using forwards and backward propagation to train a network with many stacked layers is referred to as deep learning [30, 26]. The specific deep learning architecture used in this report is a Graph Neural Network (GNN). GNNs model unstructured data represented as nodes and edges [31]. This is useful for modeling connections between entities (i.e. internet links, road networks) rather than non-relational inputs such as images. Though several GNN variants have been proposed [32, 33], the Graph Convolutional Network (GCN) is the first to gain widespread adoption [31]. As opposed to CNNs and RNNs, which may contain different block architectures in each layer, all GCN layers contain a fixed structure matching the input graph topology. Each layer updates the graph by defining its own set of node and edge attributes that are learned throughout training (Appendix Fig. A.1).

2.3 Federated Learning

A key assumption of centralized learning is that the full dataset will be available throughout the model development process. One challenge introduced by this assumption is that it requires gathering all training examples in a single location such as a server. This would require that all participating robot devices transmit raw data about their interactions with humans, which may introduce privacy concerns. These issues are critical in human-robot interaction domains where robots may be collecting sensitive user information [34].

One solution to this challenge is to maintain raw data on local devices and learn a global model by combining locally computed updates. Under this paradigm, a network of distributed phones, robots, or other *clients* downloads a common initial model from a server, fine-tunes this model based on locally available data, then transmits an updated version to the server. The server aggregates updates from participating devices into a new global model and re-distributes it to clients. This process continues iteratively until the model converges to an optimal set of parameters.

Given a set of participating clients C, each of which have a private partition of data $\mathcal{D}_c$ of size $n_c = |\mathcal{D}_c|$, the *federated optimization* objective can be formulated as:

$$\min_{\theta} G(\theta) := \sum_{c \in C} \frac{n_c}{n} g_c(\theta) \quad \text{where} \quad g_c(\theta) = \frac{1}{n_c} \sum_{i \in \mathcal{D}_c} \ell(\theta; \vec{x_i}, y_i). \tag{2.2}$$

Above, $G(\theta)$ defines the central loss function that is optimized over the distributed network of client devices, while $g_c(\theta)$ defines the local loss function for client c. Note that in this case $\ell(\theta; \vec{x_i}, y_i)$ could represent any loss function for $(\vec{x_i}, y_i)$, such as the MSE defined in equation 2.1 or cross-entropy.

One common method of optimizing the objective $G(\theta)$ is Federated Averaging (FedAvg) [35, 36]. The FedAvg algorithm (Appendix Alg. 2) iteratively refines a global model θ_G over R rounds of client-server communication by taking an average of client updates θ_c. During each communication round, the algorithm assumes a subset of all client devices will be available for training, while others may be offline. Available clients refine the central model from the previous round over E epochs of local training before transmitting updates to the server.

2.3.1 Enhancing Federated Learning

When client partitions $\mathcal{D}_c$ are formed by randomly splitting $\mathcal{D}$ into $|C|$ uniform subsets, then $\mathbb{E}_{\mathcal{D}_c}[g_c(\theta)] = G(\theta)$ and the FL objective is equivalent to centralized learning [37]. However, in practice, FL methods learn data that is distributed in a non-independent and identically distributed (*non-iid*) manner among clients [38]. Data may be non-iid due to *size imbalances* whereby clients hold varying numbers of samples. Client distributions may also be impacted by *concept drift*, where the same label y is assigned for different features $\vec{x_i}$. In a SAN setting, this may occur if two individuals say a robot position is "very appropriate" while one user is in a museum, and the other is in a home. Finally, client distributions may be impacted by *concept shift*, whereby a different label y is assigned to the same values $\vec{x_i}$. This occurs in a SAN setting if different people have conflicting preferences due to their cultural background.

To overcome non-iid data, FedAvg improvements propose a server-side aggregation modification [39, 7], or a client-side regularization penalty that prevents local models from diverging [40, 7, 41]. One mechanism for preventing local models from diverging as they learn from non-iid data is a *proximal*

loss term [40]. This term adds an L2 penalty between the local model θ and the global model θ_G to encourage local models to find solutions within the same neighborhood. During round r this term penalizes the loss function with $\frac{\mu}{2}||\theta - \theta_G^{r-1}||_2^2$, where μ controls regularization strength. Because FedProx improves performance in non-iid settings, the approach is a common baseline within the FL community [38]. When the non-iid challenges introduced by federated optimization are managed appropriately, the technique has been used successfully in a wide array of privacy-preserving application scenarios [35, 42, 43].

2.4 Continual Learning

A second challenge introduced by centralized learning is *personalization*. If a model is trained offline using a centralized dataset and the underlying distribution of $\mathcal{D}$ changes after deployment, the model may no longer perform appropriately. Therefore, maintaining performance with a new user would require retraining the full model continuously as examples from the shifting distribution are acquired [12]. This challenge of shifting data distributions is common in human-robot interaction scenarios where (1) individuals show variation among their attitudes [18], preferences [19], and physiological responses [20] to robot behaviors. Continual learning approaches solve this issue by learning a model *online* using continuously acquired data.

Given a sequence of incremental tasks, CL approaches aim to learn each without forgetting previous knowledge. This approach is designed to parallel the way that humans acquire new skills incrementally based on ongoing experiences [44]. However, because data from each task is only available temporarily, *catastrophic forgetting* of previous knowledge may occur. Therefore, a CL algorithm learns each task, consolidates knowledge into a compact representation, and retains it while learning future tasks. Provided T tasks with accompanying datasets $D^{(1:T)} = \{\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, ..., \mathcal{D}^{(T)}\}$, where each $\mathcal{D}^{(t)} \in \mathcal{D}$, a CL algorithm A^{CL} finds an optimal θ^* across tasks by mapping each task i such that:

$$\forall \mathcal{D}_i \in \mathcal{D}, \quad A_i^{CL} : < \theta^{(i-1)}, M^{(i-1)}, \mathcal{D}^{(i)}, t^{(i)} > \rightarrow < \theta^{(i)}, M^{(i)} > . \tag{2.3}$$

Here, $\theta^{(i)}$ is the model being learned at time i, $M^{(i)}$ represents an external information store used for data sampling or regularization purposes, and $t^{(i)}$ is a label for the current task.

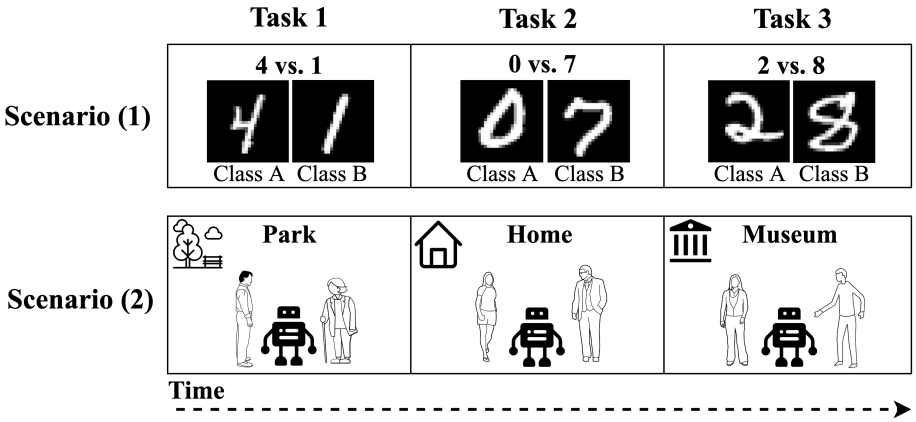

Figure 2.1: CL scenarios involving image classification (1), and social navigation (2). Scenario 1 is inspired by Fig. 1 in [45].

2.4.1 Continual Learning Scenarios

The objective optimized by algorithm 2.3 varies depending on the learning experiences the approach needs to consolidate. Therefore, several frameworks have been proposed for classifying the task structure in CL settings [8, 46, 45, 12]. One classification [8] distinguishes among structures based on whether a task-id is available at test time:

- **Task Incremental Learning (Task-IL):** Task-id provided at test time. The model is informed which task should be performed, and the approach can use task-specific parameters to solve the predictive problem.
- **Domain Incremental Learning (Domain-IL):** Task-id not provided at test time. The model is not informed which task should be performed, but task inference is not an inherent part of the prediction problem.
- **Class Incremental Learning (Class-IL):** Task-id not provided at test time *and* infer task-id. The model is not informed which task should be performed and task-inference is an inherent part of the prediction problem.

This framework is often used to categorize image classification problems. For example, Scenario (1) in Fig. 2.1 provides an example of an MNIST digit classification task. A CL algorithm could be provided (1) a task label and asked to rate which of the two classes are present (Task-IL: "Given Task 1, return

a Class A or Class B") (2) no task label and asked to rate which of the two classes are present (Domain-IL: "Given an unknown task, return Class A or Class B"), or (3) no task label and asked to rate which digit is present (Class-IL: "Given an unknown task, return a digit from $\{0, 1, ..., 9\}$"). Scenario (2) applies this framework to an HRI context. Here, a robot performs a SAN task by predicting human comfort with its location. This situation could be interpreted as a Domain-IL task in which each task contains the same predictive structure given a shifting input distribution. In this case, the social environment can be viewed as a prior that conditions the output predictions without explicitly presenting a new concept to learn. This Domain-IL scenario is relevant in long-term HRI when a robot learns to personalize to new individuals over time [12].

2.4.2 Continual Learning Strategies

Several strategies have been proposed to mitigate catastrophic forgetting during CL [44]. For example, architectural CL approaches modify task-specific network parameters [8]. These methods capture knowledge relevant across tasks while ensuring the model has capacity to learn task-specific information [47]. One architectural approach is Additive Parameter Decomposition (APD) [48]. APD consolidates knowledge by decomposing parameters $\theta^{(t)}$ into a parameter B shared across tasks and a task-specific parameter $A^{(t)}$ such that:

$$\theta^{(t)} = B \circ m^{(t)} + A^{(t)} \tag{2.4}$$

where $m^{(t)}$ is a sparse vector mask that encourages the model to learn task-adaptive knowledge [49]. This method requires an explicit task-id to load the relevant task-specific parameters. Therefore, it is useful when scenarios provide a task-id at test time (Task-IL) [48].

A second strategy for preventing forgetting is regularization. By constraining parameters such that they do not diverge from earlier tasks, it is possible to retain existing knowledge. One option is to apply an L2 constraint between the current parameters $\theta^{(t)}$ and previous parameters $\theta^{(t-1)}$ such that $||\theta^{(t)} - \theta^{(t-1)}||_2^2$ [50]. As with FedProx in FL, this *L2-transfer* strategy penalizes parameters as they diverge from an existing solution [11]. Yet, a key drawback of L2-transfer is all parameters within the network are penalized equally without regard for their importance [51]. This may over-regularize the model and slow learning on later tasks [11].

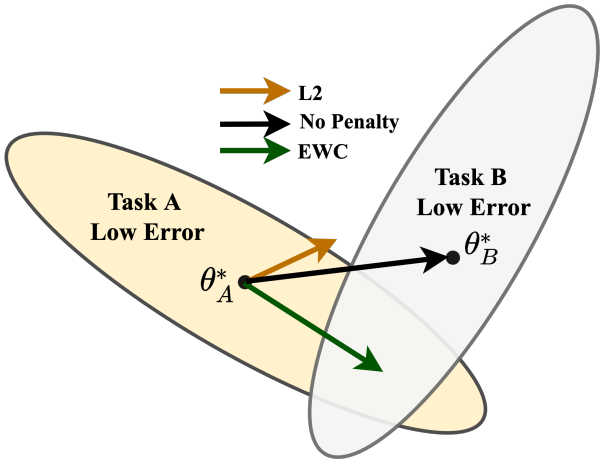

Figure 2.2: Difference between EWC, L2-transfer, and SGD while training on Task B after Task A. Diagram inspired by Fig. 1 in [11].

2.4.3 Elastic Weight Consolidation

One L2-transfer improvement is Elastic Weight Consolidation (EWC), which constrains parameters based on their importance to previous tasks [11]. EWC is motivated by the observation that θ is often over-parameterized in deep learning settings [52, 53]. Given two tasks A and B, it is therefore likely that the region of low error for Task A centered at θ^*_A intersects with the region of low error for Task B centered at θ^*_B. In a CL scenario where θ^*_A has been found, EWC attempts to find a solution for Task B near θ^*_B while remaining within the region of low error for Task A (Fig. 2.2). This is achieved by applying a quadratic penalty to each parameter and increasing the strength proportionally to its importance for previous tasks [11].

Kirkpatrick et. al. (2017) define a mechanism for determining parameter importance using a Bayesian framework [11]. Specifically, the process of optimising the parameters θ given task datasets $\mathcal{D}^{(1:T)} = \{\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, ..., \mathcal{D}^{(T)}\}$ can be framed as computing the Bayesian posterior $p\left(\theta \mid \mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(T)}\right)$ over all possible values of θ. As the model learns a new task t online, it uses the posterior from $t_{1:t-1}$ as a conditional prior while incorporating the t'th task's likelihood such that:

$$p\left(\theta \mid \mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(t-1)}, \mathcal{D}^{(t)}\right) = \frac{p\left(\mathcal{D}^{(t)} \mid \theta\right) p\left(\theta \mid \mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(t-1)}\right)}{p\left(\mathcal{D}^{(t)} \mid \mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(t-1)}\right)}. \tag{2.5}$$

In this case, only $\mathcal{D}^{(t)}$ is required while learning the t'th task because the posterior absorbs knowledge from previous tasks. Additionally, because the posterior measures θ conditioned on $\mathcal{D}^{(1:t-1)}$, it can be used to estimate parameter importance.

Unfortunately, the true posterior probability distribution is intractable [54]. EWC instead makes use of Laplace's approximation [55] to estimate the posterior using a Gaussian [1]. In particular, given a mean θ_i^* centered at the maximum a posteriori (MAP) estimate of task $i < t$ and a precision defined by the diagonal Fisher information matrix (FIM) F_i evaluated at θ_i^*, Laplace's approximation can be given as $p(\mathcal{D}^{(i)} \mid \theta) \approx \mathcal{N}(\theta; \theta_i^*, F_i)$. Provided that θ_i^* is a local minima, the FIM acts as a surrogate to the Hessian of the negative log-likelihood of the posterior distribution [54].

Taking the log of equation 2.6 and replacing the posterior with Laplace's approximation, the EWC training objective for the t'th task can be given as:

$$\theta_t^* = \text{argmin}_\theta \left\{ -\log p\left(\mathcal{D}^{(t)} \mid \theta\right) + \frac{\lambda}{2} \sum_{i=0}^{t-1} F_i \left(\theta - \theta_i^*\right)^2 \right\} \tag{2.6}$$

where λ is a parameter controlling regularization strength, and the first term is the standard negative log likelihood minimized during centralized learning.

This *offline* EWC formulation requires storing an FIM and MAP estimate for each task, which scales linearly with task size. Improved *online* EWC formulations have been proposed that use a single FIM and MAP estimate that is incrementally updated over training [56].

[1] The full derivation of Laplace's approximation to e.q. 2.5 can be found in [54, 56].

Chapter 3

Related Work

3.1 Continual Learning for HRI

In the chapter above, we highlight how CL could be used to improve robot adaptation. Recent work has explored this connection by formulating CL for affective robotics [12]. This work extends an existing CL taxonomy [8] to describe personalized socio-emotional interactions [12]:

- **New Instances (NI):** Observations in each training step relate to the same concept. In the proposed HRI framework, these *type NI* adaptations enable a robot to learn variations between of the same affective behavior (i.e. different variations of happiness facial expressions).
- **New Concepts (NC):** Observations in each training step generalize to a new concept. In the proposed HRI framework, these *type NC* adaptations enable a robot to learn new affective behavior categories (i.e. learning to distinguish happiness vs. sadness facial expressions).
- **New Instances and Concepts (NIC):** The observations contained in each training step include new instances and new concepts.

This framework provides a flexible categorization scheme based on the definition of a *concept*. Whereas the description above uses type NC and NI adaptation to describe the variation between and within expression categories, this could be adapted to explain variation between and within individuals. Therefore, whereas the Task-IL/Domain-IL/Class-IL distinction introduced in Section 2.4.1 captures macro-level aspects of the task structure, type NI/NC/NIC updates can describe more granular aspects of the robot adaptation formulation.

Based on this framework, we revisit the long-term HRI challenges identified in Section 2.1 to highlight opportunities for expansion.

- **User Adaptation:** By formulating user adaptation as a CL problem, it is possible to update the model online over interactions. However, this work primarily focuses on generalization to affective states *within* individuals. Future work can build on this formulation by *applying NC updates to handle personalization between subjects.*
- **Data Availability:** This work enhances data availability by facilitating real-world collection. However, because CL methods rely on local datasets that are encountered incrementally, performance may be poor with initial users. It may be possible to *leverage FL to accelerate learning by sharing experiences among robots.*
- **Evaluation Criteria:** By formulating online learning as a CL problem, it is possible to leverage CL metrics (to be introduced in Section 4.1.3) that capture how well an approach generalizes to future tasks. Future work could expand these metrics by *examining criteria such as time required to adapt to a new user.*

Previous work has also used CL in SAR scenarios. For example, Tjomsland et al. (2020) develop an uncertainty-guided CL approach for SAN applications [24]. This technique incrementally learns navigation preferences by approximating the posterior given in e.q. 2.5 using a Bayesian Neural Network [57, 58]. The authors show this method successfully learns human navigation preferences in an *online* manner. Together, this work and the problem formulation above show the potential for CL to address long-term HRI challenges. We now review recent developments in FL and CL before introducing a unified FCL formulation.

3.2 Federated Curvature

Shoham et at. (2019) propose using EWC regularization to improve FL convergence [51]. During each communication round r, clients exchange their MAP estimate $\hat{\theta}_{r,c}$, and precision estimate $F_{r,c}$ refined over E epochs local training. They then construct the local objective for client c during the next round as:

$$\hat{\theta}_{c,r+1} = \text{argmin}_\theta \left\{ -\log p\left(\mathcal{D}_c \mid \theta\right) + \frac{\lambda}{2} \sum_{i \in C \setminus c} F_{i,r} \left(\theta - \hat{\theta}_{i,r}\right)^2 \right\} \tag{3.1}$$

where the second term provides an importance-based regularization penalty proportional to λ [1]. This term restrains local models in a similar fashion as FedProx. However, whereas FedProx discourages local model divergence using a restrictive isotropic penalty, FedCurv does so through an elastic quadratic penalty [51]. Specifically, the difference between the c'th and i'th client's parameters is scaled by the precision estimate F_c to penalize each parameter proportionally to its significance to the model.

FedCurv has the same Bayesian underpinning as the CL case outlined in Section 2.4.3. However, an important difference with FL is that other local models have not converged. Because each client trains for E epochs-per-round rather than to task convergence, the MAP estimates $\hat{\theta}_{c,r}$ serving as the mode of Laplace's approximation are not at a local minima, and therefore approximate the true MAP estimate $\theta^*_{c,r}$ such that $\hat{\theta}_{c,r} \approx \theta^*_{c,r}$. This reduces FedCurv performance in cases with low E [51].

[1] In practice, this objective is re-arranged to optimize for communication efficiency [51]. We present an unoptimized version to draw an explicit parallel with EWC loss in e.q. 2.6

3.3 Personalized Federated Learning

A second line of emerging work leverages FL for *personalization*. These methods learn a global model while also adapting to user-specific information [59, 60, 61]. For example, a recently proposed personalized pain detection approach learns a CNN feature extractor through FL and applies personalization by training a fully-connected output layer with user-specific data [62]. Results show FL performance matches centralized learning, but indicate that FL can be subject to *negative transfer*, whereby model averaging impedes performance for some subjects. Existing FL personalization approaches do not focus on this client interference problem, suggesting it is an avenue for future work [59].

There appears to be a distinction between FL evaluations that assume adversarial non-iid conditions, and FL personalization evaluations, which examine baselines under more realistic assumptions. For example, FedAvg performs poorly in highly non-iid cases [37, 51], but its performance has been robust in emerging personalized FL evaluations [62, 61, 59]. Additionally, attempts to bolster FedAvg performance with user-specific parameters have seen limited success: Rudovic et al. (2021) report a 1% performance improvement with FedAvg+personalization over the FedAvg baseline [62], while a second personalized FL method offers performance within the margin of error of FedAvg+finetuning [61].

Despite the promise of personalized FL, current techniques are not designed to handle non-stationary data distributions that contain *concept drift* and *concept shift* over time. Leveraging frameworks from CL, one early work proposes combing FedAvg with a change detection technique for detecting shifts in the input distribution during training [63]. However, as noted in a recent personalized FL review [59], temporal adaptability remains "an open direction" in FL personalization – a challenge continual learning is well-positioned to address.

3.4 Federated Continual Learning

Yoon et al. (2020) propose a Federated Continual Learning (FCL) paradigm combining FL and CL [9]. Under this learning scenario, a network of distributed clients each learns a series of local tasks. Each task may be available to a subset of other clients (at a past, present, or future task-ID; Fig. 3.1) or all local tasks may be mutually disjoint. This paradigm introduces a new opportunity for *inter-client knowledge transfer*, whereby experiences from previous tasks can be shared among clients to improve learning. However, it also introduces a challenge of *inter-client interference*, in which experiences aggregated from other clients accelerate catastrophic forgetting. FCL approaches maximize knowledge transfer while minimizing interference to learn experiences that are distributed spatially (FL) and temporally (CL).

Yoon et al. (2020) propose an architectural approach for solving the FCL problem [9]. This technique decomposes a global model θ_G into a set of client-specific base parameters B and task-adaptive parameters A that learn knowledge about each local task. This approach encourages knowledge transfer by enabling clients to exchange task-adaptive parameters and weight them locally via an attention mechanism. The decomposed global parameter can be given as:

$$\theta_c^{(t)} = B_c^{(t)} \circ m_c^{(t)} + A_c^{(t)} + \sum_{i \in C \setminus c} \sum_{j<t} \alpha_{i,j}^{(t)} A_i^{(j)} \tag{3.2}$$

where $m_c^{(t)}$ is a sparse vector mask used to reduce the number of non-zero components of $B_c^{(t)}$ before transmission, and $\alpha_{i,j}^{(t)}$ is a learnable attention mechanism used to adaptively weight the knowledge of other clients. This method, called FedWeIT, extends the APD approach developed for CL (e.q. 2.4) to leverage FL and inter-client knowledge transfer (Appendix Alg. 3).

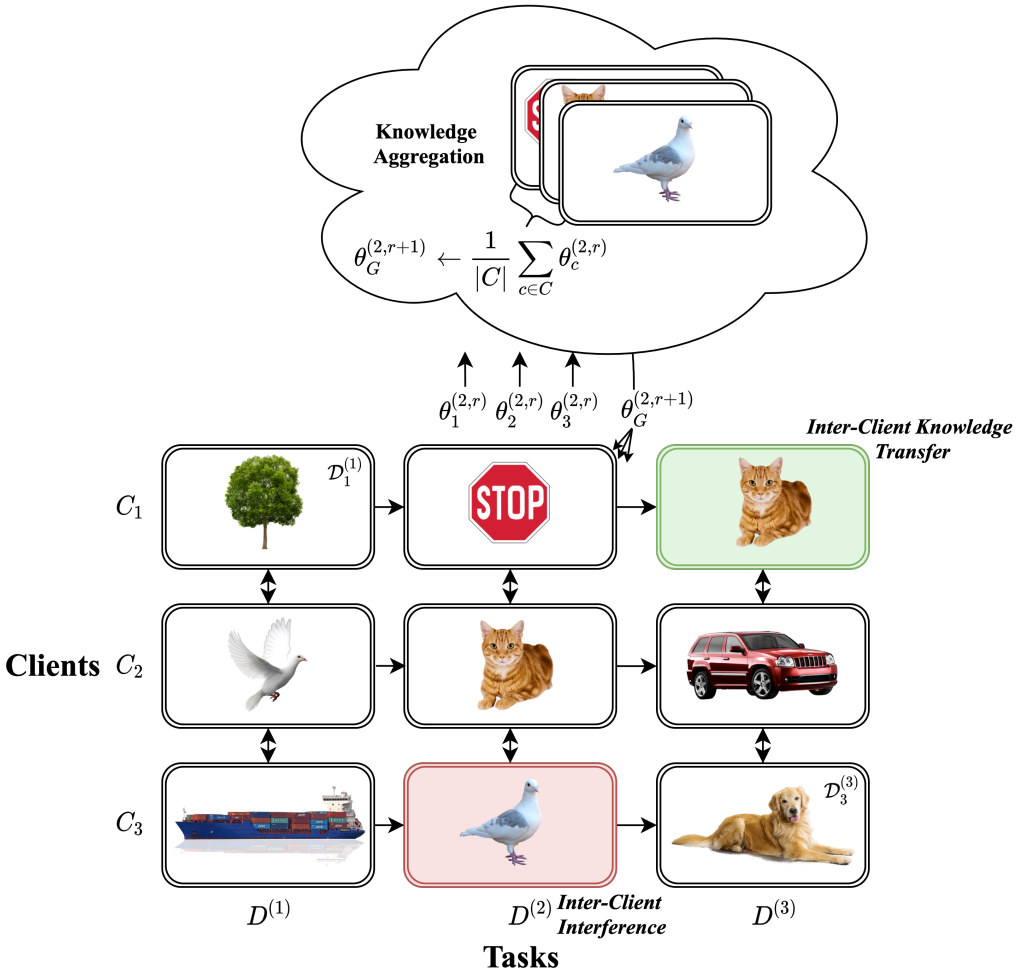

Figure 3.1: FCL leaning scenarios assume that knowledge is distributed spatially over clients and temporally over tasks. This shows the knowledge aggregation process during communication round r of Task 2. Each client c only has access to their partition of $\mathcal{D}_c^{(2)}$. The client attempts to remember information from the previous Task 1 and generalize to the future Task 3. Knowledge transfer occurs when C_1 uses information about the cat class acquired from C_2 during Task 2 as it learns Task 3. Interference occurs when knowledge that C_2 learned during Task 1 is forgotten while C_3 learns Task 2, which contains similar information.

Chapter 4

Methodology

We begin this chapter with an FCL for HRI problem formulation. We then identify an existing HRI dataset for a proof-of-concept evaluation. Finally, we establish a centralized learning benchmark on the dataset. This provides a baseline for decentralized learning evaluations in Chapters 5 and 6.

4.1 Problem Formulation

4.1.1 Scenarios

Long-term HRI scenarios follow different structures with varying numbers of participants and sessions [4]. Therefore, a first step for formalizing FCL for HRI is defining a series of concrete *learning scenarios* that specify the structure of interactions. These scenarios encode the structure of tasks and the number of robots (or *clients*) interacting in parallel. It is helpful to consider both the spatial (FL) and temporal (CL) aspects of interactions.

- **FL Characteristics** The spatial arrangement of interactions is determined by the number of robots deployed in parallel. For example, multiple socially assistive robots may be deployed to a care home, where they communicate with one another as they learn a new skill over a series of long-term interactions with residents. The number of robots in use maps to the number of clients (C) in an FL formulation.
- **CL Characteristics** The temporal arrangement of interactions can be framed using the CL taxonomies presented in Sections 2.4.1 and 3.1. For example, temporal interactions may be structured into tasks based on recognizing new socio-emotional attitudes and behaviors [12], personalizing to new individuals [64], or adapting to a user over a series of repeated tutoring sessions [15]. Regardless of the specific structure, the principal concern of a CL algorithm is the *concept drift* that occurs in the data over interactions. Concept drift instances may be split into discrete task boundaries (Task-IL), or the algorithm may need to detect these (Domain-IL/Class-IL) [63, 45].

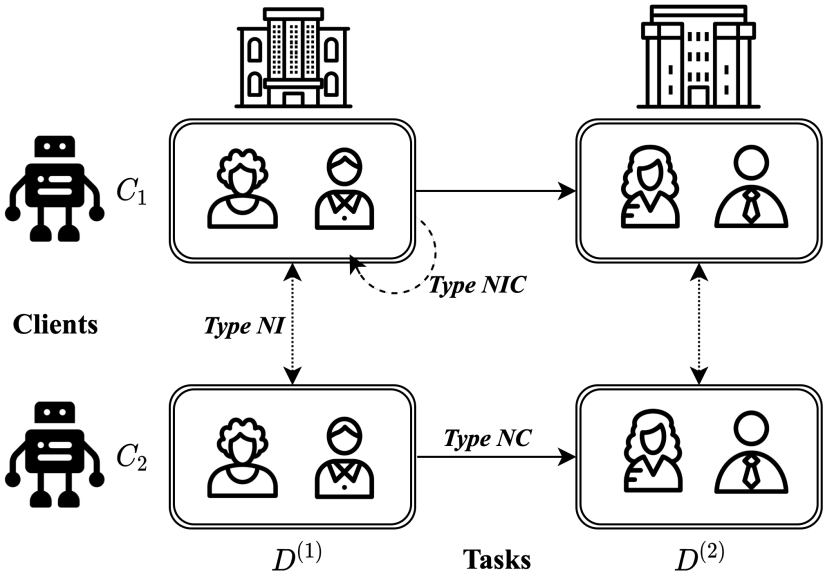

Figure 4.1: Scenario A, where robots personalize to a new groups many-to-many over tasks.

We can combine FL and CL characteristics to describe spatial and temporal adaptation. Given an HRI scenario with *between-subjects variability*, we can frame generalizations to new individuals as NC updates. Similarly, new sessions within the same person can be framed as less challenging NI updates. Combining the two, we use NIC updates to describe new contexts that contain new individuals and new sessions from existing individuals. In an FCL context, NC updates may be distributed over *clients* if each robot interacts with a different user, while NC updates may also be distributed over *tasks* if a robot personalizes to multiple users over time. We provide two examples to illustrate this generalization of CL content types to FCL.

Scenario A. A network of robots interacts with a new cohort of individuals during each task. Within each cohort, interactions are many-to-many between people and robots. In the care home example, several robots may be stationed at multiple locations (cafeteria, gift shop, etc.) within the building. Residents may interact with each other at different points in time. After several months in care home A, the robots move to care home B, where the same multi-user personalization process repeats (type NC update). In this case, the local dataset available to each robot contains data from multiple individuals over multiple interactions (type NIC updates). Because each robot processes data from the same individuals, this constitutes NI updates between clients.

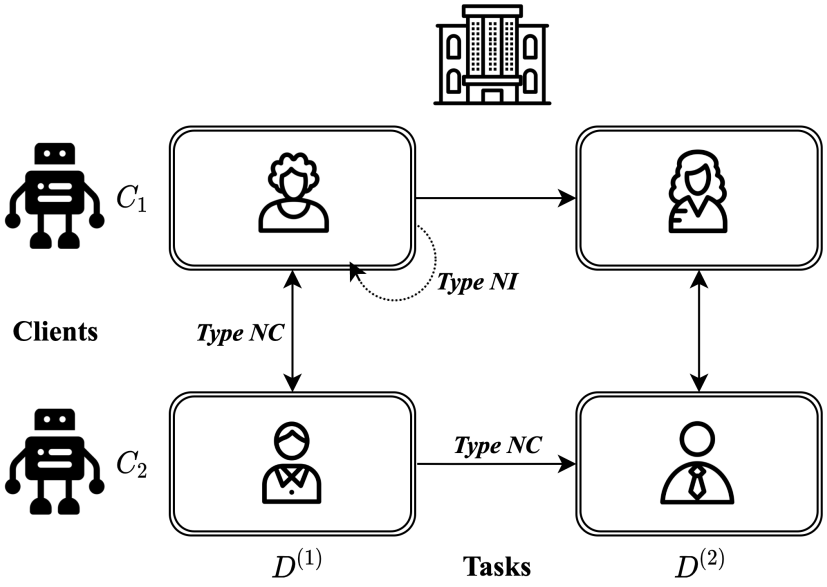

Figure 4.2: Scenario B, where robots personalize to a new individual one-to-one over tasks.

Scenario B. A network of robots each personalize to a new individual over tasks. Here, humans and robots interact one-to-one over multiple sessions. In the care home environment, robots may be stationed in the private room of several residents, where they adapt to the individual over several months. Robots later move to a new set of rooms, where they each personalize to a new resident. Here, the local dataset available to each robot contains information from a single individual over multiple sessions (type NI updates). However, each robot and task contains an NC update describing generalization to a new participant. Given these scenarios, we now consider how an ideal FCL approach should perform.

4.1.2 Performance Dimensions

In an FCL context, it is crucial to examine several dimensions to gain a holistic understanding of performance. For example, one naive approach for solving the scenarios above would be to train a separate model for each client-task pairing and load the model dynamically during test time. This Single Task Learning (STL) method may provide strong performance (high *adaptation quality*). However, it would also require many rounds of human-robot feedback to provide sufficient data.

To improve the *adaptation time*, we may enhance STL by running a local CL algorithm on each robot. Here, initial participants still provide many feedback examples, but later participants benefit from fine-tuning an initialized model. However, if the first participants return to interact with the robot in later tasks, the robot may require additional interactions to recover the initial adaptation quality due to *catastrophic forgetting*. A further drawback of CL and STL is that they are unable to leverage experiences from other robots. Therefore, to facilitate *knowledge sharing* among robots, we may enable each local CL algorithm to share parameters through FL. Thus, an FL-CL approach such as FedWeIT can be used[1]. However, if each participant's preferences differ due to *concept shift*, adding FL could introduce *inter-client interference* and reduce adaptation quality.

Other trade-offs to consider include *privacy* and *model overhead*. Because STL, Local-CL, and FL-CL employ *decentralized learning* they all improve user privacy. However, it is important to consider overhead factors including model size, client-server communication costs, and energy consumption [8, 38].

Desiderata

Given these considerations, we form a set of desiderata describing how an ideal FCL algorithm should function. We can describe each dimension (1) in terms of the behavior that the robot should demonstrate, and (2) the underlying algorithmic function:

1. **High Adaptation Quality.** After a series of long-term interactions, the robot is well-tuned to the specific needs of the individual. An ideal FCL approach achieves this with strong performance on the relevant predictive task by satisfying its training objective (i.e. low MSE, high F1-score, etc.).

2. **Minimal Adaptation Time.** The robot quickly adjusts to an individual's interaction style after only a few feedback sessions with minimal delay. An ideal FCL approach achieves this by (1) generalizing to new predictive tasks with strong performance, (2) requiring a small dataset size, and (3) converging quickly during training on a new participant's dataset.

[1] FL-CL denotes a method that uses FL and CL to solve the FCL training scenario.

3. **Effective Knowledge Sharing.** The robot effectively utilizes knowledge and experience from other robots to achieve high adaptation quality with a faster adaptation time. An ideal FCL approach achieves this by (1) maximizing inter-client knowledge transfer and (2) minimizing inter-client interference.

4. **Privacy Preserving.** The robot minimizes the information required to achieve high adaptation quality. An ideal FCL approach achieves this by (1) transmitting model parameters instead of data (FL), and (2) not requiring permanent storage of data on the robot (CL). Note that this criterion is satisfied automatically based on the assumptions of FCL.

5. **Minimal Model Overhead.** The robot's financial, energy, and connectivity requirements are minimized. An ideal FCL approach achieves this by (1) reducing the information transmitted during communication rounds, (2) reducing the number of trainable and static model parameters required, and (3) minimizing extra computational steps.

In the next section, we outline the specific evaluation methodology and metrics used to quantify these high-level FCL design objectives.

4.1.3 Evaluation & Metrics

Recall that in a centralized learning paradigm, $\mathcal{D}_{train}$ is used to optimize the model, while $\mathcal{D}_{test}$ measures its generalization to new data. An FCL algorithm requires a more granular approach, as the $\mathcal{D}$ has now been partitioned into a $C \times T$ grid of client-task datasets $\mathcal{D}_c^{(t)}$. Performance on a local client-task dataset can be estimated by splitting $\mathcal{D}_c^{(t)}$ into a training partition $Tr_c^{(t)}$ and test partition $Te_c^{(t)}$ (as well as a validation partition $Tv_c^{(t)}$ if necessary). However, because datasets are distributed spatially and temporally, evaluating a single global model over all local partitions becomes more challenging. Therefore, we discuss FL and CL related aspects of the evaluation independently before outlining the overall FCL evaluation process [9].

Under *federated evaluation* in FL, the training and testing process involves (1) refining the global model θ_G^r over E epochs of local training, (2) sending updated client models θ_c^r to the server, (3) computing a new aggregate model θ_G^{r+1}, and (4) distributing θ_G^{r+1} to clients for evaluation on $Te_c^{(t)}$ [7]. An aggregate performance measure is computed by averaging client-specific performance. Because θ_G^{r+1} is evaluated across clients, this provides an estimate of generalization across users.

Evaluation of CL involves measuring temporal performance of the algorithm over tasks. In addition to current task performance, an evaluation can measure how well approaches remember previous tasks, and how well the generalize to new ones [8]. Temporal CL performance is measured through a training-test performance matrix $P \in \mathbb{R}^{T \times T}$, where rows denote training datasets $Tr^{(i)}$ available to the algorithm during each task, and columns denote the corresponding set of test datasets $Tr^{(j)}$ [8] (Fig. 4.3). Given this matrix structure, the *Average Mean Squared Error* (AMSE) over tasks can be given as $\frac{1}{T}\sum_{i=1}^{T} P_{i,i}$. Here, we adapt the average accuracy metric proposed in [8] to fit the regression task outlined in Section 4.2. We can further extend this AMSE metric to measure *Backward Transfer* (BWT) and *Forward Transfer* (FWT).

P	$Te^{(1)}$	$Te^{(2)}$	$Te^{(3)}$
$Tr^{(1)}$	$R^{(1,1)}$	$R^{(1,2)}$	$R^{(1,3)}$
$Tr^{(2)}$	$R^{(2,1)}$	$R^{(2,2)}$	$R^{(2,3)}$
$Tr^{(3)}$	$R^{(3,1)}$	$R^{(3,2)}$	$R^{(3,3)}$

Figure 4.3: Train-test accuracy matrix P, inspired by the one provided on page 18 of [8]. Cyan and white used for BWT, white used for FWT, grey used for FWT.

.

BWT measures how well an approach retains knowledge from previous tasks [8]. It is computed by measuring the performance difference between the previous task and the current task:

$$BWT = \frac{\sum_{i=2}^{N}\sum_{j=1}^{i-1}(P_{i,j} - P_{j,j})}{\frac{N(N-1)}{2}} \tag{4.1}$$

Because we measure BWT in the context of MSE, a negative BWT indicates performance is improving over tasks. Therefore, results with a lower BWT are favored.

To measure generalization to unseen tasks, we can compute FWT as the mean of the grey elements in Fig. 4.3 such that:

$$FWT = \frac{\sum_{i=1}^{j-1} \sum_{j=1}^{N} P_{i,j}}{\frac{N(N-1)}{2}} \tag{4.2}$$

Grey entries represent test datasets whose accompanying training datasets have not been used for model optimization. With MSE, an approach provides better generalization capacity if its FWT score is closer to zero.

These FL and CL specific considerations can be unified into a FCL evaluation approach. During each training round on a given task t, models follow the federated evaluation process to (1) fine-tune client-specific models using $Tr_c^{(t)}$, (2) compute and redistribute a new global model, and (3) evaluate the new global model on all client-task test datasets. Aggregate AMSE, BWT, and FWT metrics are then computed after averaging the CL train-test performance matrix elements across clients. Appendix Fig. A.4 provides a detailed schematic of this process. We can now summarize high-level user considerations, algorithm desiderata, and metrics in Table 4.1.

4.1.4 Algorithms

Given this formulation, we now establish a set of alternate FCL approaches. Building on the design discussion in Section 4.1.2, one way of classifying an FCL algorithm is whether it leverages STL, CL, or FL-CL during training:

- **Single Task Learning (STL):** A STL approach trains a separate model on each local client-task dataset.
- **Local (CL):** A *local* approach trains a separate model on each client using CL. This extends STL by sharing the model among tasks.
- **FL-CL:** A FL-CL approach enables clients to communicate using FL as they solve their local CL problem. This extends STL and CL algorithms by enabling clients to share a global model.

As a model is shared among more local client-task partitions, this increases the opportunity to share learning experiences, but also requires measures to prevent interference. Therefore, FL and CL-specific design improvements may be combined to aid learning. This introduces a dimension of whether an FCL algorithm leverages *architectural* or *regularization* enhancements.

Table 4.1: FCL for long-term HRI: algorithm performance dimensions.

Dimension	User	Algorithm	Metrics
Adaptation Quality	• How well does the robot adapt to my behaviors after months of interaction?	• How well does the model perform at the end of the personalization process?	• AMSE
Adaptation Time	• How often do I need to give my robot feedback? • How many days do I need to wait for the robot adapt to me?	• How well does the model generalize to a new person with no feedback examples? • How much data does the model require to generalize to a new person? • How many training rounds does the model take to converge?	• FWT • AMSE as a function of dataset size • Convergence speed in loss plot
Knowledge Sharing	• Same considerations as adaptation time.	• **Inter-client knowledge transfer:** How well does the approach transfer knowledge and experience among clients? • **Inter-client interference:** How much does the knowledge of one client interfere with the knowledge of another?	• FWT • BWT
Privacy	• What information do I need to share?	• What information does the model require to reach sufficient adaptation quality?	• N/A
Model Overhead	• How much does the robot cost? • How often do I need to charge the robot?	• How much storage does the robot require? • What is the model's communication overhead?	• Number of model parameters • Model communication overhead • CL/FL specific metrics [7, 8]

- **Architectural:** These enhancements solve the FCL scenario by adding new parameters for each client-task dataset. The APD method provided as CL background is one example of a local architectural strategy, while FedWeIT is an example of an FL-CL architectural strategy. STL is also an architectural strategy because it functions by dynamically loading model parameters. Because a task-id is required to load relevant model parameters, architectural approaches assume a Task-IL scenario.
- **Regularization:** These enhancements solve the FCL scenario by preventing models from diverging from one another as they train on local client-task datasets. For example, whereas L2-Transfer and EWC may be used to prevent local CL model divergence, FedProx and FedCurv may be used to prevent FL model divergence.

These dimensions provide a flexible taxonomy for describing a specific FCL training method. For example, an FL-CL algorithm FedProx-APD combines FedProx for FL communication with APD for local model training.

4.2 Dataset

We now wish to connect this formulation to a concrete long-term HRI learning problem. The first step of this process is to select an HRI benchmark dataset. However, as discussed previously, an ongoing challenge in the HRI community is developing a corpus of large annotated datasets [65]. Because establishing such a dataset is outside the scope of this work, a review of existing HRI benchmarks was conducted. Three criteria were used for the selection:

- **Dataset Size:** The dataset should contain as many examples as possible. This is because neural networks require sufficient data to converge, and the FCL partitioning process reduces the dataset size available to each local SGD solver by a factor of factor $C \times T$ [9].
- **Task Structure:** The dataset should motivate a supervised learning predictive task. Supervised learning is preferable because many long-term HRI problems involve annotated examples from human subjects, which motivates a supervised learning problem [4].

- **Learning Scenarios:** The dataset should motivate FCL learning scenarios outlined in Section 4.1.1. Specifically, the dataset should contain participant annotations that are non-iid due to (1) different situational characteristics (*concept drift*), (2) size imbalances, and (3) annotation disagreements (*concept shift*).

Conference proceedings were reviewed for datasets that fit these criteria using the keywords *dataset* and *benchmark*. The search included proceedings from IROS (2018 [66], 2019 [67]), RO-MAN (2019 [68], 2020 [69]), HRI (2019 [70], 2020 [71]), and FG (2019 [72], 2020 [73]), and an additional Google Scholar keyword search. The *SocNav1* dataset matched the criteria the most closely [10].

4.2.1 SocNav1 Dataset

SocNav1 is a benchmark for evaluating socially-aware robot navigation models [10]. It consists of 9280 labeled examples gathered from 12 participants using a desktop-based annotation tool. Participants were presented a static social scenario including humans, robots, objects, and interactions, and were instructed to *"assess the robot's behavior in terms of the disturbance caused to humans"* on a continuous range from 0 to 100 (Appendix Fig. A.3). Details of the data collection protocol are provided in [10].

Situations in each vignette were randomized to present a variety of configurations to participants. The quantity, orientation, and position of humans, objects, and robots were randomized. To assess annotation reliability, the set of 9280 annotations was gathered from a subset of 5735 unique scenarios. The SocNav1 authors evaluated inter-rater reliability using the kappa coefficient [74] and found values ranging from 0.61 to 0.71, which indicate "substantial" agreement. Additionally, these commonly annotated scenes can be used to construct FCL evaluation scenarios with *non-iid concept drift* by assigning participants who have annotated the same scenarios to different client-task datasets. The top row of Fig. 4.4 shows the distribution of situational characteristics across rated scenarios. Scenes vary, which enables constructing scenarios with *non-iid concept shift*. Similarly, the bottom row of Fig. 4.4 shows the distribution of ratings for each participant. Participant dataset sizes also vary, which enables constructing FCL scenarios with *non-iid dataset imbalances*.

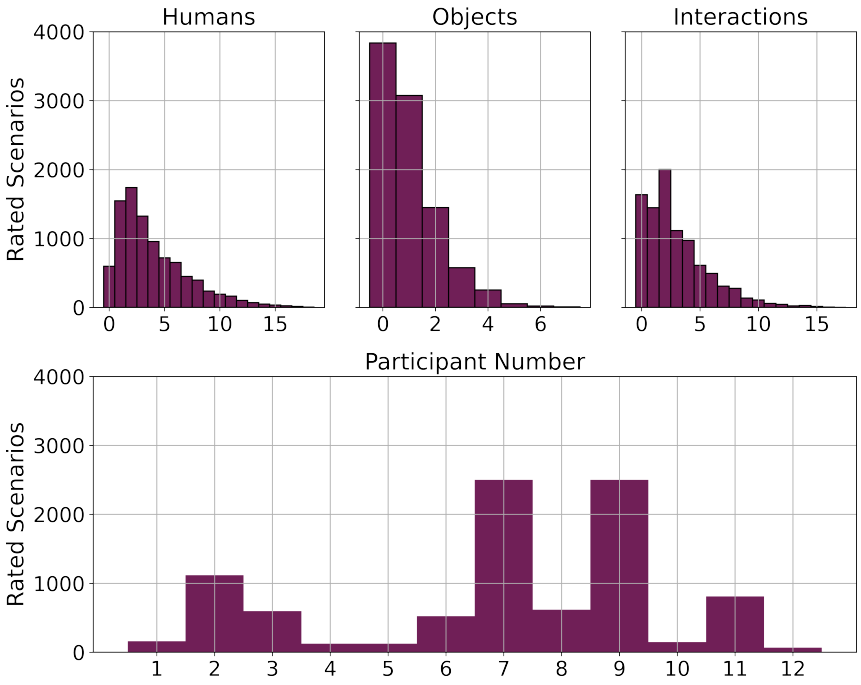

Figure 4.4: Distribution of SocNav1 scenario characteristics (top) and ratings provided by each participant (bottom).

4.2.2 Data Representation

SocNav1 represents annotated social scenes in a graph structure. Each scene occupies an 8×8 meter Euclidean plane centered at the origin. Human, robot, and object entities are positioned at an x, y coordinate at an orientation given in degrees. Each entity is stored with a unique identifier, and human-human and human-object interactions are specified via a tuple containing the identifier of each entity. Because scenes occupy a Euclidean plane, they can be augmented by mirroring scenes over the Y-axis and assigning the same participant rating [10]. This augmentation strategy is common in computer vision problem domains [75]. Further detail regarding data representation and augmentation can be found in [23].

		Task 1	Task 2
	Client 1	1471, 315, 316	1778, 381, 381
	Client 2	1470, 315, 316	1775, 381, 381
+ Aug.	**Client 1**	2942, 315, 316	3556, 381, 381
	Client 2	2940, 315, 316	3550, 381, 381

Table 4.2: Scenario A: Client-task train, validation, test dataset sizes with and without data augmentation.

		Task 1	Task 2	Task 3	Task 4
	Client 1	111, 24, 24	781, 168, 168	417, 90, 90	86, 19, 19
	Client 2	86, 18, 19	365, 78, 79	1750, 375, 375	431, 92, 93
	Client 3	1750, 375, 375	103, 22, 23	46, 10, 10	565, 121, 122
	Client 1	222, 24, 24	1562, 168, 168	834, 90, 90	172, 19, 19
+ Aug.	**Client 2**	172, 18, 19	730, 78, 79	3500, 375, 375	862, 92, 93
	Client 3	3500, 375, 375	206, 22, 23	92, 10, 10	1130, 121, 122

Table 4.3: Scenario B: Client-task train, validation, test dataset sizes with and without data augmentation.

4.2.3 Learning Scenarios

Annotations from SocNav1 can be used to construct FCL evaluation scenarios. Recall from Section 4.1.1 that Scenario A involves two cohorts of participants interacting with multiple robots in a many-to-many setting. This scenario can be formulated from SocNav1 by grouping annotations from the 12 participants into two tasks and two clients. Specifically, Task 1 can be formed by shuffling annotations from participants 1-6 and splitting them among $\mathcal{D}_1^{(1)}$ and $\mathcal{D}_2^{(1)}$, while Task 2 can be formed by shuffling annotations from participants 7-12 and splitting them among $\mathcal{D}_1^{(2)}$ and $\mathcal{D}_2^{(2)}$. Table 4.2 provides the client-task dataset sizes when the SocNav1 dataset is partitioned using this approach with a 70/15/15 % train/test/validation split.

Similarly, recall that Scenario B involves each participant and robot interacting one-to-one. This can be constructed by assigning annotations from each SocNav1 participant to an individual client-task dataset. In particular, we split the 12 participants into 3 clients and 4 tasks. In an HRI context, this structure simulates three robots communicating as they interact with four sets of individuals one-on-one. Table 4.3 provides the client-task dataset sizes when SocNav1 is partitioned using this approach.

4.3 Benchmark Evaluation

In this section, we develop a GNN model and benchmark its performance with centralized learning. This serves to (1) verify that the dataset [10] and accompanying model [23] perform as expected, (2) establish a suitable hyperparameter configuration, and (3) provide a decentralized learning baseline.

4.3.1 Model Architecture

Because SocNav1 represents scenes in a relational format, GNNs are a natural architecture for predicting human navigation preferences. Therefore, Manso et al. (2020) evaluate several GNN architectures for predicting social appropriateness ratings [10, 23]. As shown in Table 1 of [23], the top-performing model produces a test MSE of 0.031. This maps to ± 17.6 points on the 0-100 annotation scale, which is approaching the inter-subject annotation variation of ± 14.8 points calculated using the subset of scenarios rated by multiple participants. While considering which architecture reported in [23] to adapt for FCL, other factors are relevant in addition to performance. Because FL and CL evaluations aim to isolate differences in training methodology [7], experiments often make use of a simple baseline model [9]. This reduces architecture-related factors that could confound analysis. A simple architecture is also favored for FL because model size impacts storage and communication costs [41]. Given these considerations, the DGL+GCN[2] model was selected for further benchmark testing. We select this architecture because (1) it provided robust train MSE on the reported evaluation, and (2) the GCN is the most basic GCN architecture.

4.3.2 Centralized Learning Evaluation

Apart from the best configuration in Table 1 of [23], the SocNav1 authors do not report test MSE and hyperparameter settings for each architecture. The authors were contacted to clarify, but these details were not available. Therefore, it was necessary to replicate the hyperparameter search with the DGL+GCN architecture to (1) establish a test MSE, and (2) identify the corresponding hyperparameter settings. This evaluation followed the same protocol reported by the SocNav1 authors. A grid of settings varying the number of network layers, hidden layer size, weight decay (γ), and mini-batch size was randomly sampled (Appendix Table A.1). Ten sampled configurations were trained with $\alpha = 5e^{-3}$ for 400 epochs each using a pre-partitioned

[2]Here, DGL denotes Deep Graph Library, a commonly used PyTorch GNN implementation. [76]

$\mathcal{D}_{train}$, $\mathcal{D}_{valid}$, $\mathcal{D}_{test}$ split provided by the SocNav1 authors [10]. The size of $\mathcal{D}_{train}$ was augmented using the method described in Section 4.2.1. Based on trends in $\mathcal{D}_{valid}$ MSE observed from randomly sampled configurations, an additional fine-tuned architecture combining the best settings was also evaluated.

Evaluation Results

The fine-tuned DGL+GCN architecture provided the lowest test MSE of 0.0296, with the full results provided in Appendix Table A.2. This outperforms the top test MSE of 0.03173 reported by the SocNav1 authors. These results confirm that GNNs successfully estimate participant ratings provided in the SocNav1 dataset. This is further supported by an R2-score of 0.74 achieved on $\mathcal{D}_{test}$, which indicates that GNN predictions account for roughly 75% of the variation in participant ratings. In addition to reasonable MSE and R2, results also indicate that the fine-tuned DGL+GCN architecture converges to a low MSE quickly (Appendix Fig. A.2). This provides a centralized learning baseline for decentralized FCL evaluations. Finally, this experiment also establishes a backbone GCN architecture with 8 layers and 32 hidden units per layer. We retain this architecture throughout experiments reported in Chapters 5 and 6.

Chapter 5

Federated Continual Learning

We now examine the trade-offs of the algorithms discussed in Section 4.1.4. These experiments (1) evaluate how well each approach fits the HRI desiderata established in Section 4.1.2, and (2) provide a detailed algorithmic comparison. We leverage insights from these experiments to motivate a new regularization-based FCL strategy in Chapter 6.

5.1 Motivation

Though algorithms outlined in Section 4.1.4 have been benchmarked extensively [41, 7, 8], these evaluations involve highly non-iid partition schemes. For example, the FCL evaluation in [9] constructs client-task datasets along class boundaries of (1) CIFAR-100 and (2) a combination of 8 image classification datasets. This evaluation differs from long-term HRI scenarios in several ways. First, whereas the evaluation above provides *balanced* local dataset sizes, in an HRI context, the client-task datasets may be *imbalanced*. Second, while the evaluation above assumes a bijective mapping between images and labels, in HRI scenarios, labels are prone to *concept shift* from annotator subjectivity. Finally, the overall size of HRI datasets is smaller than image classification benchmarks. Therefore, we conduct a comparison of methods reported in [9] on the long-term HRI scenarios described in Section 4.1.1.

5.2 Experiments

5.2.1 Experiment 1

Experiment 1A compares FCL approaches in Scenario A with and without augmentation. Scenario A models many-to-many interactions between two robots and two cohorts of participants. Specifically, this scenario was constructed by assigning SocNav1 participants 1-6 to Task 1, and participants 7-12 to Task 2, then splitting annotations from each task evenly into two clients (Table 4.2). Because client-task dataset sizes are uniform, non-iid dataset imbalances are minimized. This experiment is also designed to introduce more complex adaptation between tasks and within client-task datasets by grouping participants into two cohorts. While learning new samples *within* Task 1 and Task 2, clients combine knowledge across 6 different participants, which constitutes *Type NIC adaptation*. While transitioning *between* Task 1 and Task 2, clients generalize to a new group of participants, which constitutes *Type NC adaptation*. Conversely, *Type NI adaptation* occurs between clients because data from the same participants is shared among clients during each task.

Experiment 1B provides the same comparison in FCL Scenario B. This benchmark models one-to-one interactions with *Type NC* adaption between tasks and clients, and *Type NI* adaptation within client-task datasets. This scenario was constructed by assigning each SocNav1 participant to a client-task dataset, with three clients and four tasks total. Here, local datasets are imbalanced, with augmented training partitions ranging from 92 to 3500 samples each. This setting emulates non-iid size imbalances. Though Scenario B presents a less complex HRI scenario, we posit this benchmark will produce worse performance because small local datasets are likely to pose a significant impediment to local SGD convergence. We mitigate this effect by assigning client-task datasets with the most samples to different tasks and clients, as shown in Table 4.2. Given this, we form the following scenario performance hypotheses:

- **H1.1:** Training dataset augmentation will produce lower AMSE scores.
- **H1.2:** Scenario A will produce lower AMSE scores than Scenario B.

Based on results reported in Table 1 of [9], we can predict how FCL algorithms outlined in Section 4.1.4 may perform. The FCL authors show STL and Local-CL approaches outperform FL-CL approaches. The authors also show architectural strategies such as FedWeIT perform well. Therefore, we form the following algorithm performance hypotheses:

- **H1.3:** STL and Local-CL approaches will achieve lower AMSE than FL-CL approaches.
- **H1.4:** Architectural strategies will achieve lower AMSE than regularization strategies.

5.2.2 Experiment 2

Though Experiment 1 compares with and without augmentation, this may not isolate how approaches learn from varying amounts of training data. Therefore, Experiment 2 conducts a data ablation analysis by training each algorithm using 20, 40, 60, 80, and 100 percent of the augmented training dataset. We construct client-task datasets by modifying each partition in Scenario A such that a fraction of $Tr_c^{(t)}$ is included, and the full ($Tv_c^{(t)}$, $Te_c^{(t)}$) are included. Scenario A was used for this analysis because client-task datasets become prohibitively small in under 20% and 40% partitions of Scenario B. This experimental setup is effective for understanding the *adaptation time* dimension of performance, as some methods may be able to improve *adaptation quality* more easily as the size of client-task datasets increases. Because this analysis is not reported in [9], we evaluate the basic hypothesis:

Ablation Performance:

- **H2:** All algorithms will converge to a lower AMSE as the training dataset size increases.

5.3 Methods

The algorithms compared in experimental evaluations include STL, FedWeIT, and all combinations of <*Local*, *FedProx*, *FedAvg*>-<*APD*, *EWC*, *SGD*>, except for Local-SGD. We select this subset of methods because (1) they are reported in [9], (2) they span all dimensions of the STL/Local-CL/FL-CL, architecture/regularization taxonomy, and (3) they are widely-used methods. The FedWeIT implementation provided by the FCL authors [1] was ported from TensorFlow to PyTorch to support compatibility with the existing DGL+GCN backbone model provided with the SocNav1 dataset. This implementation offers a simple FL simulation environment based on Python multithreading. The FedWeIT authors did not provide code for their baseline implementations. Therefore, relevant baselines were implemented and incorporated into the FedWeIT simulation architecture. In all cases, the same DGL+GCN architecture established in Section 4.3 was used.

Experimental Setup

Recall that in Experiment 1A, the benchmark scenario has two tasks and two clients, where each task contains a group of 6 SocNav1 participants. In Experiment 1B, the benchmark contains four tasks and three clients, where each task and client consists of a different SocNav1 participant. Client-task datasets were partitioned into a $Tr_c^{(t)}$, $Tv_c^{(t)}$, $Te_c^{(t)}$ split in a 70/15/15 ratio (as shown in Tables 4.2 and 4.3). Performance on $Tv_c^{(t)}$ was used for hyperparameter experiments, model testing, and initial sanity checks. Final results on all experiments are provided using performance on $Te_c^{(t)}$. Experiments 1 and 2 were run with FCL settings ($R = 75$, $E = 1$, $C_r = C$) and ($R = 60$, $E = 1$, $C_r = C$) respectively. Because Experiment 2 requires evaluating each algorithm five separate times, one for each training dataset fraction, we set $R = 60$ to keep the computation time feasible. We keep $E = 1$ as reported by the FCL authors. Selecting all clients each training round diverges from the evaluation in [9]. However, we do this to control for the effects of dropping one of the three clients and conduct a follow-up analysis in later experiments.

All experiments use $\alpha = 5e^{-3}$, $\gamma = 1e^{-4}$, and a mini-batch size of 32 samples. An Adam optimizer with learning rate decay was used, where the α is reduced by a factor of 2 for every 5 consecutive epochs with no validation loss decrease. Training continues at the minimum learning rate of $1e^{-5}$ rather than using early stopping to synchronize training among clients. We set $\mu = 5e^{-2}$ for FedProx, $\lambda = 5e^{-2}$ for EWC, $\lambda_1 = [1e^{-2}, 1e^{-4}]$ and $\lambda_2 = 1e^{-4}$ for FedWeIT.

[1] https://github.com/lilujunai/federated-continual-learning

Algorithm	Evaluation Metrics			Parameters	
	AMSE	BWT	FWT	Static	Trainable
Baselines					
STL	0.0377	**-0.0044**	0.4801	0	15362
+ Aug.	0.0351	0.0007	0.3174	0	15362
FedAvg-SGD	0.0320	-0.0001	0.0401	0	7681
+ Aug.	**0.0285**	0.0067	**0.0288**	0	7681
FedProx-SGD	0.0339	-0.0013	0.0411	7681	7681
+ Aug.	0.0313	0.0054	0.0331	7681	7681
Regularization					
Local-EWC	0.0331	**-0.0014**	0.0406	30724	7861
+ Aug.	0.0329	0.0037	0.0365	30724	7861
FedAvg-EWC	0.0301	-0.0007	0.0378	30724	7861
+ Aug.	**0.0287**	0.0075	**0.0291**	30724	7861
FedProx-EWC	0.0339	-0.0010	0.0438	38405	7861
+ Aug.	0.0301	0.0068	0.0306	38405	7861
Architectural					
Local-APD	0.0412	0.1458	0.1638	0	23300
+ Aug.	0.0393	0.2144	0.2121	0	23300
FedAvg-APD	0.0414	0.1849	0.2673	0	23300
+ Aug.	0.0445	0.0731	0.2576	0	23300
FedProx-APD	0.0412	0.2076	0.2038	7424	23300
+ Aug.	0.0418	0.1986	0.2157	7424	23300
FedWeIT	0.0394	0.1221	0.9179	29700	23332
+ Aug.	0.0410	0.2862	0.2695	29700	23332

Table 5.1: Experiment 1A results. Top performance across baselines is given in bold and underlined, while second best performance is given in bold.

5.4 Results

5.4.1 Experiment 1

Table 5.1 shows the AMSE, BWT, FWT, and parameter count of each baseline in Experiment 1A, while Table 5.2 shows the same metrics for Experiment 1B. These results are grouped by the CL strategy, with STL, FedAvg-SGD, and FedProx-SGD serving as baselines. Across configurations, data augmentation reduced AMSE and FWT, but increased BWT. This confirms **H1.1** and indicates that augmentation improves performance, but makes it more difficult to retain previous knowledge. Additionally, algorithms perform better in Scenario A than Scenario B across AMSE, BWT, and FWT, confirming **H1.2**. Overall, this high-level comparison indicates that larger dataset size imbalances and smaller local client-task datasets reduces performance across metrics.

Algorithm	Evaluation Metrics			Parameters	
	AMSE	BWT	FWT	Static	Trainable
Baselines					
STL	0.0937	0.0119	0.5362	0	30724
+ Aug.	0.0615	0.0110	0.4175	0	30724
FedAvg-SGD	0.0425	0.01174	**0.0408**	0	7681
+ Aug.	0.0426	**-0.0031**	0.0483	0	7681
FedProx-SGD	0.0473	0.0062	0.0442	7681	7681
+ Aug.	0.0431	**-0.0024**	0.0508	7681	7681
Regularization					
Local-EWC	0.0615	0.0152	0.0786	61448	7681
+ Aug.	0.0497	0.0134	0.0819	61448	7681
FedAvg-EWC	0.0425	0.0136	**0.0397**	61448	7681
+ Aug.	**0.0410**	0.0003	0.0446	61448	7681
FedProx-EWC	0.0464	0.0109	0.0429	69129	7681
+ Aug.	**0.0419**	-0.0019	0.0495	69129	7681
Architectural					
Local-APD	0.0816	0.0846	0.1767	0	39176
+ Aug.	0.0521	0.1159	0.1838	0	39176
FedAvg-APD	0.0858	0.1628	0.1541	0	39176
+ Aug.	0.0480	0.2230	0.5101	0	39176
FedProx-APD	0.0897	0.1609	0.1669	7424	39176
+ Aug.	0.0495	0.1625	0.2263	7424	39176
FedWeIT	0.1044	0.2061	0.3618	89100	39272
+ Aug.	0.0959	0.2259	0.6610	89100	39272

Table 5.2: Experiment 1B results. Note that some methods have more parameters here than in Table 5.2 due to model expansion over tasks.

Within Experiments 1A and 1B, FL-CL baselines and FL-CL methods using an EWC CL strategy perform the best. Specifically, FedAvg-SGD produced the lowest AMSE in Experiment A (0.0285), while FedAvg-EWC produced the lowest AMSE in Experiment B (0.0410). STL, FedWeIT, and architectural Local-CL strategies have a higher AMSE, poor forward generalization capacity (high FWT), and poor recall of previous tasks (high BWT). Because FL-CL approaches (with the exception of FedWeIT) outperform STL and Local-CL, we reject **H1.3**. Likewise, because regularization strategies outperform architectural ones, we reject **H1.4**.

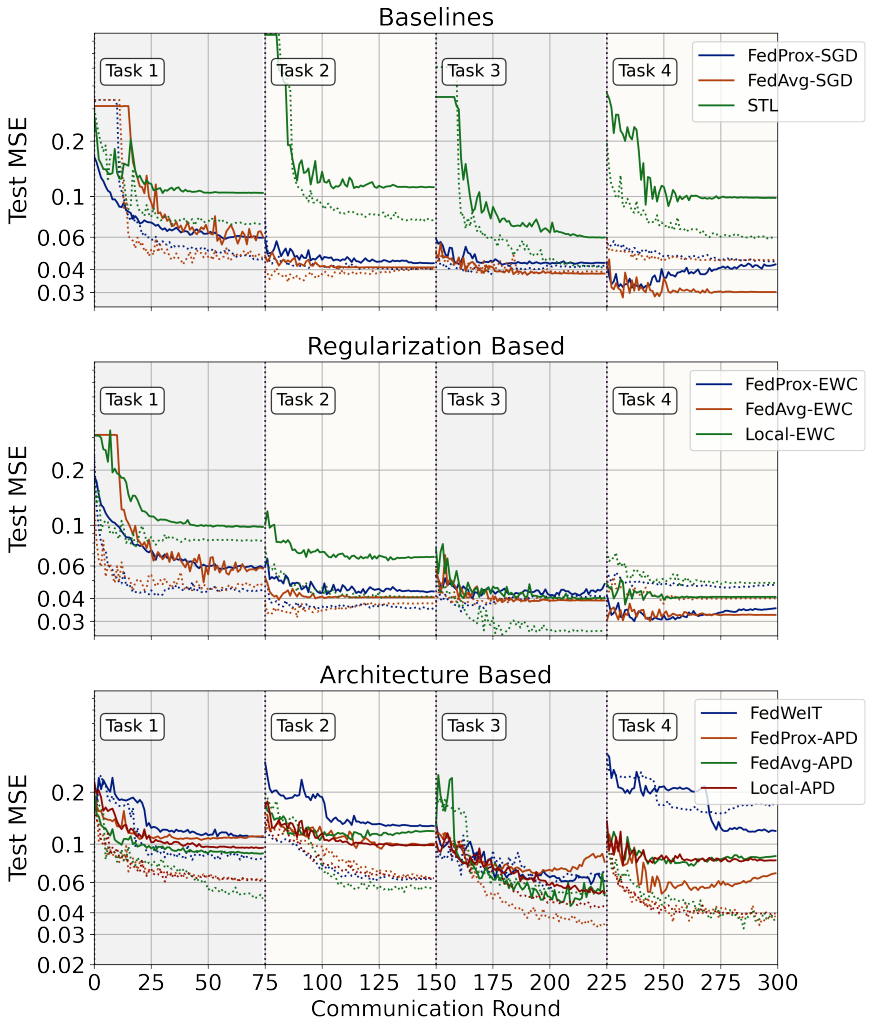

Figure 5.1: Experiment 1B test MSE over communication rounds. Dotted and solid lines show with and without data augmentation, respectively. The y-axis is given in a log scale to highlight performance differences.

Fig. 5.1 shows the convergence of each algorithm over communication rounds in Experiment 1B. MSE of STL and architecture-based CL strategies plateau at a higher value as task training rounds conclude. These methods also show a spike in MSE at the beginning of training on each task as the model learns a new set of task-adaptive parameters. Conversely, we observe that Local-EWC, FL with SGD, and FL with EWC tend to produce a lower final MSE at the end of training on each task. Within the regularization-based strategies, we also observe a distinct difference between Local-CL and FL-CL methods. Specifically, the MSE of Local-EWC converges more slowly and plateaus at a higher MSE in Task 1 and Task 2 before reaching similar performance with FL-CL approaches in later tasks.

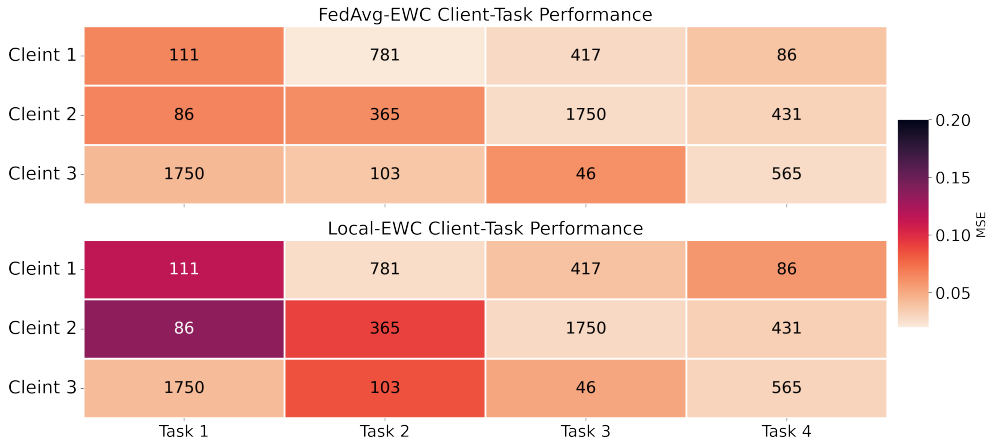

Figure 5.2: Test MSE on client-task datasets at the end of training on each task without augmentation. Client-task training dataset size is shown in cells.

Fig. 5.2 compares client-task dataset MSE between FedAvg-EWC and Local-EWC to examine the performance difference above in greater detail. During the first task, both CL and FL-CL reach low MSE on $Te_3^{(1)}$ with 1750 training examples. However, whereas FedAvg-EWC is able to share this knowledge with $Te_2^{(1)}$ and $Te_1^{(1)}$, clients 1 and 2 of Local-EWC are required to learn models independently. Therefore, it is difficult to achieve strong performance on local partitions with few samples. This Local-EWC performance deficiency continues on proceeding tasks but becomes less pronounced as Local-EWC encounters more training data. Therefore, *knowledge sharing* may account for improved *adaptation time* and *adaptation quality* with the SGD/EWC FL-CL methods.

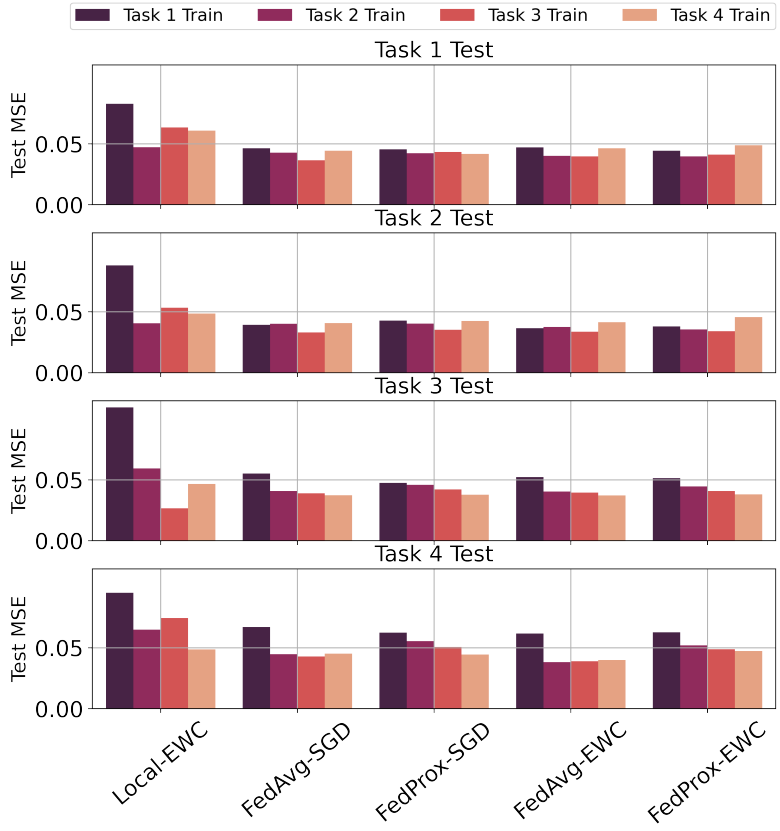

Figure 5.3: Test MSE on each task over training rounds. Visualization shows transposed P, where $Te_c^{(t)}$ occupies rows and $Tr_c^{(t)}$ occupies columns.

We examine *knowledge sharing* by comparing performance on $Te^{(t)}$ before and after training on $Tr^{(t)}$. Specifically, Fig. 5.3 visualizes performance on each test dataset over training tasks for the top-5 performing methods [2]. If performance on $Te_c^{(t)}$ improves *before* task t is available, and continues to improve after t, this indicates knowledge transfer has occurred. Conversely, if performance on $Te_c^{(t)}$ deteriorates after training on task t, this indicates inter-client interference has occurred. Results show that Local-EWC provides less knowledge transfer compared with FL-CL methods. Additionally, results show that FedAvg-EWC and FedAvg-SGD are more effective at knowledge transfer than FedProx-EWC and FedProx-SGD. We discuss these findings in further detail in Chapter 6.

[2]We provide a forgetting analysis plot with the full entries of P in appendix Fig. A.5.

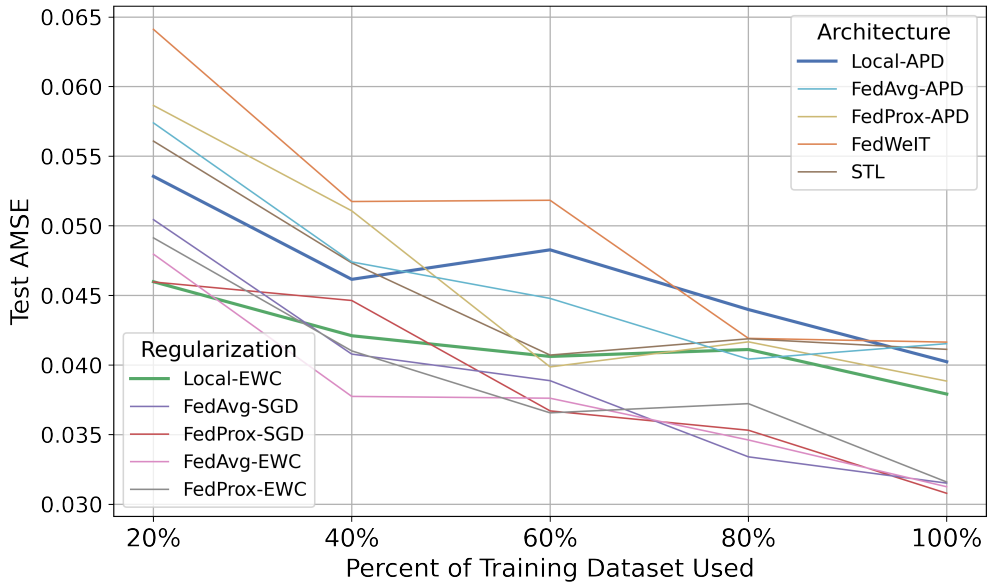

Figure 5.4: Test AMSE as a function of augmented training dataset size.

5.4.2 Experiment 2

Fig. 5.4 shows the AMSE for each method in Experiment 2. Results confirm **H2** and show that performance improves across methods as the training dataset size increases. However, results also suggest that some methods are more effective at leveraging smaller fractions of the training dataset. Specifically, performance on Local-CL models is comparably strong when 20% of the training data used. As the amount of training data scales, performance improves at a slower rate than FL-CL approaches. For example, at 100% dataset size, Local-EWC reaches the same performance that regularization FL-CL approaches do at 60%. These results support the interpretation that regularization FL-CL approaches improve *adaptation time* by improved data efficiency in the form of *knowledge sharing*.

5.5 Analysis

Overall, a key finding of these experiments is that decentralized learning can achieve similar performance to the centralized learning baseline in long-term HRI settings. The best AMSE in Experiments 1A and 1B is 0.0285 and 0.0410, respectively. This is within the range of the 0.0296 MSE on $\mathcal{D}_{test}$ achieved in the centralized benchmark. Within decentralized learning, experiments indicate FL-CL approaches with SGD/EWC provide the best *adaptation quality*. Because these approaches demonstrate low FWT, quick loss convergence, and AMSE improvement as dataset size increases, we also conclude FL-CL approaches improve *adaptation time*. By examining client-specific performance during Task 1 (Fig. 5.2) and performance on test partitions over training tasks (Fig. 5.3), we isolate effective *knowledge sharing* as the likely mechanism for this performance advantage. Additionally, the strong performance of FedAvg-SGD in these evaluations replicates recent work reporting FedAvg as providing strong performance in FL personalization settings [62].

Our results also show that the long-term HRI scenario is important. Performance is better in Experiment 1A compared with 1B, likely due to larger local client-task dataset sizes in 1A. This is supported by Fig. 5.2, which shows MSE is inversely correlated with dataset size. Additionally, our finding that architectural strategies perform poorly diverges from results in [9]. This may also be due to the evaluation scenario, as the original FCL evaluation in [9] contained larger local dataset sizes. These larger local datasets may have required additional parameters from architectural methods to learn appropriately.

Chapter 6

Elastic Transfer

Chapter 5 demonstrates that several FL-CL algorithms perform well in long-term HRI scenarios. Here, we leverage these findings to propose an improved algorithm. We then conduct a second series of experiments to evaluate its performance characteristics. Results show the proposed *Elastic Transfer* (ET) approach outperforms existing methods in this evaluation.

6.1 Motivation

In the experiments above, we observe FedAvg-SGD, FedAvg-EWC, FedProx-SGD, and FedProx-EWC demonstrate the best FCL performance. Though these methods achieve comparable ASME, there are key differences in their *knowledge sharing* capacity. Specifically, we note that in Fig. 5.3 these methods show different MSE on $Te^{(4)}$ over training rounds. Whereas FedAvg-SGD $Te^{(4)}$ leverages transfer from $Tr^{(2)}$ to converge quickly, this process is more gradual with FedProx-SGD. Likewise, FedAvg-EWC benefits from the same $Tr^{(2)}$ knowledge transfer, while FedProx-EWC does so less effectively. We also observe that FedProx/FedAvg-EWC provides a performance enhancement over FedProx/FedAvg-SGD. These characteristics may be explained by the EWC motivation discussed in Section 2.4.3. In particular, the isotropic penalty introduced by FedProx may be overly restrictive, thereby slowing learning and limiting inter-client knowledge transfer. In contrast, we observe *importance-based local regularization* provided by EWC enhancing performance on future tasks. Compared with Local-EWC we see that EWC regularization is especially effective in an FL-CL setting.

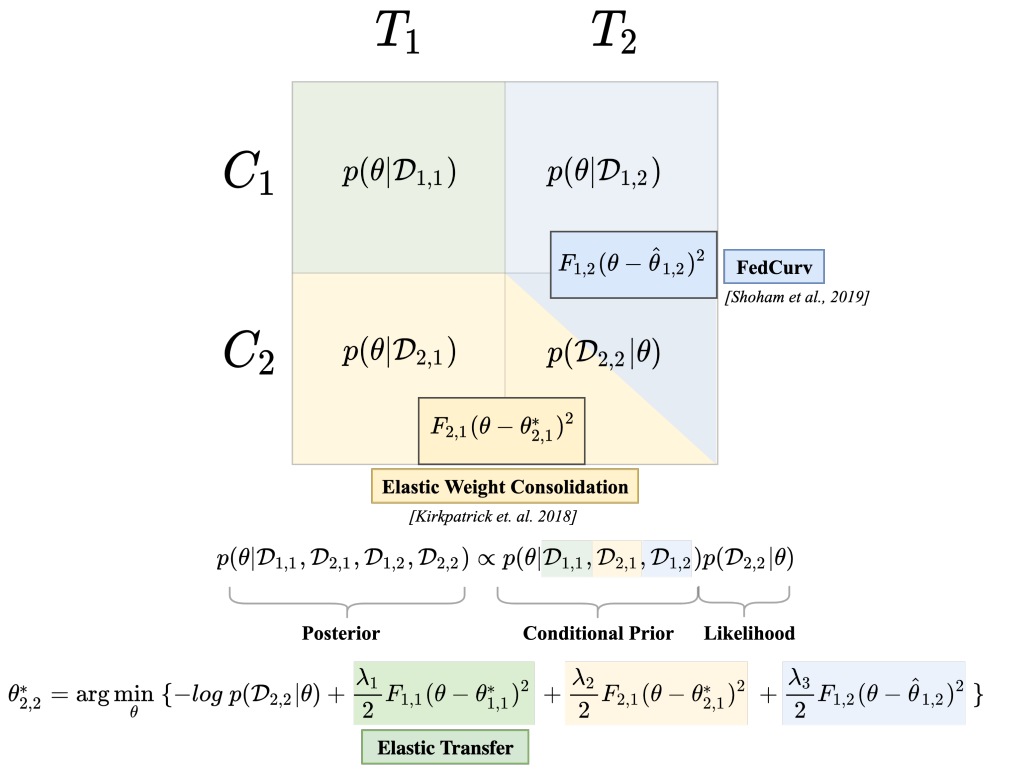

Figure 6.1: Schematic diagram of training on the local dataset $\mathcal{D}_{2,2}$. F represents FIMs (precision estimate) while θ^* represent MAP estimates. In this case, MAP estimates for $\theta^*_{2,1}$ are optimal because Task 1 has already converged to a solution during the previous task. However, $\hat{\theta}_{1,2}$ estimates are a poor approximation at the beginning of Task 2 training because the task has not been solved.

While FedAvg-EWC regularizes local training based on previous tasks *from the same client*, this does not consider parameter importance estimates derived from *other clients*. This may cause models to overwrite knowledge from other's previous local tasks during training and impede knowledge retention. Recall from Section 3.2 that the knowledge interference problem has been addressed in an FL-specific context through FedCurv. This method uses EWC importance-based regularization to improve convergence stability between clients and was shown to (1) prevent local model divergence, while (2) avoiding the limitations of FedProx. Therefore, it may be possible to combine the insights from EWC in the CL case and FedCurv in the FL case to develop an improved FCL approach.

6.2 Formulation

Fig. 6.1 summarizes how FedCurv and EWC can be applied to the FCL problem. This diagram illustrates the process of training a local model on $\mathcal{D}_{2,2}$[1] in a scenario with two clients and two tasks. Extending the EWC argument presented in Section 2.4.3, we formulate the process of learning $\mathcal{D}_{2,2}$ as incorporating the likelihood term $p\left(\mathcal{D}_{2,2} \mid \theta\right)$ into the conditional prior of other client-task partitions $p\left(\theta \mid \mathcal{D}_{1,1}, \mathcal{D}_{2,1}, \mathcal{D}_{1,2}\right)$. As with EWC, we can apply Laplace's approximation to make the conditional prior tractable. We approximate this term using three cases:

- **Case 1: Previous task, same client**. Here, we approximate $p\left(\theta \mid \mathcal{D}_{2,1}\right)$ using $\theta^*_{2,1}$, $F_{2,1}$ as MAP and precision estimates for Laplace's approximation (beige in Fig. 6.1). Because $\theta^*_{2,1}$ is a previous task on the same client, this is simply Offline-EWC. Here, we expect $\theta^*_{2,1}$ to serve as a reasonable MAP estimate because the parameters represent a local minimum after training on Task 1.
- **Case 2: Same task, different client**. In this case (blue in Fig. 6.1), when we approximate $p\left(\theta \mid \mathcal{D}_{1,2}\right)$, we use $\hat{\theta}_{1,2}$, $F_{1,2}$ from other clients as an MAP and precision estimate. As described in Section 3.2, $\hat{\theta}_{1,2}$ acts as a rough approximation of the true MAP value because $\hat{\theta}_{1,2}$ is also being trained in Task 2. However, if the number of local epochs per round E is sufficiently high, we can expect $\hat{\theta}_{1,2} \approx \theta^*_{1,2}$.
- **Case 3: Previous task, different client**. In this case (green in Fig. 6.1), we approximate $p\left(\theta \mid \mathcal{D}_{1,1}\right)$ using the same process as local EWC. Here, $\theta^*_{1,1}$, $F_{1,1}$ act as MAP and precision estimates for Laplace's approximation, and we assume that $\theta^*_{1,1}$ is at a local minimum due to convergence on a previous task.

In practice, the relative importance of conditioning on $p\left(\theta \mid \mathcal{D}_{2,1}\right)$, $p\left(\theta \mid \mathcal{D}_{1,2}\right)$, $p\left(\theta \mid \mathcal{D}_{1,1}\right)$ will likely vary as a function of the specific training scenario. However, we predict that the diagonal element (Case 3) may provide the most benefit because (1) it provides generalization across tasks and clients, and (2) its MAP estimate has converged to a local minimum. Given this formulation adapting importance-based regularization to FCL problems, we now develop an *Elastic Transfer* (ET) algorithm showing how this can be applied in practice.

[1]Here, we switch client-task notation from $\mathcal{D}_c^{(t)}$ to $\mathcal{D}_{c,t}$ to make the notation for an optimal model $\theta^*_{c,t}$ more legible.

6.3 Algorithm

The formulation above enables an FCL algorithm to (1) reach a *consensus* on which parameters are important *across clients and previous tasks*, and (2) *penalize local changes* to these parameters. Thus, models exchange knowledge through FL, while also performing *elastic transfer* of parameter importance estimates. In contrast to the existing FedWeIT approach that introduces additional task-adaptive parameters, this algorithm *consolidates knowledge* to retain previous experiences while generalizing to new ones.

A naïve implementation of the 2×2 scenario above requires storage and communication of $\theta^*_{c,t}$ and $F_{c,t}$ that scales $\mathcal{O}(C \times T)$ as a function of scenario size. We can reduce this to be linear as a function of C by leveraging the online adaptation used to optimize EWC. This reformulation applies Laplace's approximation to the full posterior rather than each likelihood term individually [54, 56]. Specifically, the Online-EWC variant stores a single FIM estimate F^*_c and re-centered MAP estimate $\theta^*_{c,t-1}$ to solve each local CL problem on client c. The FIM estimate F^*_c is computed as a running sum of fisher information matrices over tasks. With reference to Fig. 6.1, this enables scaling the beige region to any T while maintaining a constant storage requirement of F^*_c, $\theta^*_{c,t-1}$. Leveraging this optimization, we construct the local loss on the client-task partition $\mathcal{D}_{c,t}$ as:

$$-\log p\left(\mathcal{D}_{c,t} \mid \theta\right) + \frac{\lambda_1}{2} \sum_{i \in C} F^*_i (\theta - \theta^*_{i,t-1})^2 + \frac{\lambda_2}{2} \sum_{i \in C \setminus c} \hat{F}_i (\theta - \hat{\theta}_{i,t})^2 \tag{6.1}$$

where for each client c, $F^*_c := \sum_{i=1}^{t-1} F_i$ is the running sum of previous tasks' precision estimates. The first term uses λ_1 to weight *refined* estimates F^*, θ^* from previous tasks, while the second uses λ_2 to weight *rough* estimates of $\hat{\theta}$, $\hat{F}$ currently being optimized across clients. With respect to Fig. 6.1, the first term applies Online-EWC (baige) when $c = i$, and Elastic Transfer when $c \neq i$ (green). The second term applies FedCurv (blue) when $c \neq i$. Algorithm 13 demonstrates how e.q. 6.1 can be optimized across clients and tasks.

Overall, this method remains space and communication inefficient. In particular, clients must exchange FIM's and MAP estimates many-to-many, which becomes burdensome as C scales. It may be possible to leverage the same storage and communication optimization employed by FedCurv to reduce transmission overhead. We leave this to future work and instead focus on evaluating this basic implementation strategy on long-term HRI scenarios.

Algorithm 1: ELASTIC TRANSFER

Input: Client-task datasets $\{\mathcal{D}_{c,t}\}_{c\in C, t\in\{1,\dots,T\}}$, Epochs-per-round E, Rounds-per-task R, Set of clients C, Number of tasks T

Output: Optimized global model θ^*_G

Initialize global model θ_G
for $t \leftarrow 1$ ***to*** T **do**
  if $t > 1$ **then**
    for $c \in C$ **do**
      Update F^*_c, $\theta^*_{c,t-1}$ from previous task ▷ `Online-EWC update.`
      Send F^*_c, $\theta^*_{c,t-1}$ to other clients ▷ `Elastic Transfer step.`
  for $r \leftarrow 1$ ***to*** R **do**
    Select available clients $C_r \subseteq C$
    for $c \in C_r$ **do**
      $\hat{\theta}_{c,t},\ \hat{F}_{c,t} \leftarrow TrainLocal(\theta_G, E)$ ▷ `Optimize equation 6.1.`
      Send $\hat{\theta}_{c,t}, \hat{F}_{c,t}$ to other clients ▷ `FedCurv information.`
    $\theta_G \leftarrow \sum_{c\in C_r} \frac{1}{|C_r|}\hat{\theta}_{c,t}$ ▷ `FL model aggregation.`
return $\boldsymbol{\theta}^*_G$

6.4 Experiments

In these experiments, we first isolate regularization-based performance variation by comparing baselines in FL and CL-specific training contexts. Then, we conduct an FCL evaluation using Scenario B with data augmentation. In the FCL analysis, we vary the presence and strength of regularization penalties 1) within the same client (EWC), 2) across clients (FedCurv), and 3) across previous tasks on different clients (ET).

6.4.1 Experiment 3

In Experiment 3, we construct a CL scenario by arranging SocNav1 partitions into 1 client and 6 tasks. We then compare the performance of SGD, L2T($\mu = 5e^{-2}$), Offline-EWC($\lambda = 5e^{-2}$), and Online-EWC($\lambda = 5e^{-1}, 5e^{-2}, 5e^{-3}$) over 75 epochs of training per task. The goal of this analysis is to (1) perform a sanity check confirming that Online and Offline EWC achieve similar performance, (2) compare EWC performance with Local-SGD (no CL strategy) and L2-Transfer (L2 penalty), and (3) vary EWC regularization strength. We compare λ and μ at $5e^{-2}$ across L2T, Offline-EWC, and Online-EWC to control for regularization strength. To avoid severe data imbalances, which impacted performance in Fig. 5.2, we interleave client-task datasets according to size. Specifically, 6 client-task datasets were formed by sorting the 12 client-task datasets used in Scenario B by size, then pairing the largest and

smallest partitions with one another. Given this scenario, we wish to understand how well EWC methods, SGD, and L2T learn each new task while remembering previous ones. Recall from the EWC background discussed in Section 2.4.3 that L2T impedes learning by adding an overly strict penalty. Conversely, though SGD can easily learn new tasks, it has no mechanisms preventing catastrophic forgetting [51, 11]. Therefore, we hypothesize:

- **H3.1:** The AMSE of L2T will be higher than SGD and EWC.
- **H3.2:** SGD will provide the lowest AMSE and highest BWT.

6.4.2 Experiment 4

In Experiment 4, we construct an FL scenario with 6 clients and 1 task by using the SovNav1 partitions from Experiment 3 as client datasets. We compare performance of FedAvg, FedProx($\mu = 5e^{-2}$), and FedCurv($\lambda = 5e^{-1}, 5e^{-2}, 5e^{-3}$). Likewise, the goal of this analysis is to (1) establish baseline performance of FedCurv compared with FedAvg (no FL strategy), and FedProx (L2 penalty), and (2) vary FedCurv regularization strength. To compare knowledge consolidation across CL and FL, we also fix FedCurv regularization strengths to the same values of $5e^{-1}, 5e^{-2}, 5e^{-3}$ used in Experiment 3. A further aim of this experiment is to benchmark FL performance in less optimistic training conditions. Specifically, we vary epochs per round E between 1, 5, and 10 (Exp 4A) and dropped clients per round D between 0, 2, and 4 (Exp 4B). We set D to 2 during Exp. 4A and E to 5 during Exp. 4B. Experiment 4 was conducted over 40 communication rounds, which is lower than previous evaluations, but necessary given the larger E configuration [2]. Previous FedCurv results show the method performs better as E increases [51]. This is because MAP estimates become increasingly reliable as the local training rounds progress. Additionally, the importance-based regularization method used by FedCurv may provide additional stability in scenarios where D is high. Therefore, we hypothesize that:

- **H4.1:** FedCurv MSE relative to FedAvg and FedProx will decrease as E increases.
- **H4.2:** FedCurv MSE relative to FedAvg and FedProx will decrease as D increases.

We predict FedCurv performance relative to FedAvg and FedProx because varying D and E may introduce baseline variation across methods.

[2] All experiment settings match Chapter 5 unless otherwise indicated.

6.4.3 Experiment 5

In Experiment 5, we examine performance of the Elastic Transfer objective (e.q. 6.1) on the Scenario B benchmark. This experiment varies three regularization penalties: λ_1^{EWC}, λ_1^{ET}, and λ_2, which control the strength of EWC, ET, and FedCurv, respectively. We decompose λ_1 in the first term of e.q. 6.1 into λ_1^{EWC} for cases when $c = i$, and λ_1^{ET} for cases when $c \neq i$. We provide additional granularity here to separate the effects of EWC and ET regularization. As in e.q. 6.1, λ_2 controls FedCurv regularization strength. We vary the settings of λ_1^{EWC}, λ_1^{ET}, and λ_2 between 0 (no regularization), $5e^{-2}$, and $5e^{-1}$, with FedProx and FedAvg as baselines. We make this evaluation more realistic than Experiment 1 by dropping 1 of the 3 clients each training round and setting E to 5. Because dropping 1 of 3 clients may introduce variation in training dynamics, we randomly select the ordering of dropped clients at the beginning of the experiment and maintain this throughout runs. Additionally, we use $R = 25$ instead of the $R = 75$ used in Experiment 1 to limit the computational burden of using $E = 5$.

Because EWC, ET, and FedCurv regularization strength have not been compared in an FCL evaluation[3], it is difficult to predict how λ_1^{EWC}, λ_1^{ET}, and λ_2 strength will interact. However, we expect that ET regularization may improve FCL convergence by preventing clients from overwriting important knowledge from previous tasks. If this is successful, this should improve knowledge transfer, reduce inter-client interference, and result in lower overall MSE. Therefore, we predict that:

- **H5.1:** As ET regularization increases, AMSE will decrease.
- **H5.2:** As ET regularization increases, BWT will decrease.
- **H5.3:** As ET regularization increases, FWT will decrease.

[3]Yoon et al. (2020) report a FedCurv+EWC baseline in Exp. 2, but do not propose ET or vary regularization strengths [9].

Algorithm	Evaluation Metrics			Parameters	
	AMSE	BWT	FWT	Static	Trainable
Online-EWC $\lambda = 5e^{-1}$	**0.0284**	0.0073	**0.0376**	15362	7681
Online-EWC $\lambda = 5e^{-2}$	0.0293	**0.0070**	0.0386	15362	7681
Online-EWC $\lambda = 5e^{-3}$	0.0285	0.0083	0.0381	15362	7681
Offline-EWC $\lambda = 5e^{-2}$	0.0312	0.0071	0.0407	92172	7681
L2T $\lambda = 5e^{-2}$	0.0306	0.0078	0.0413	46086	7681
SGD	0.0287	0.0079	0.0385	0	7681

Table 6.1: Results of Experiment 3. Further visualizations are reported in Appendix Fig. A.6.

6.5 Results

6.5.1 Experiment 3

Results of Experiment 3 comparing CL regularization strategies are given in Table 6.1. AMSE, BWT, and FWT across methods are similar, making it difficult to reach definite conclusions. However, from the trends present, Online-EWC appears to out-perform Offline-EWC and L2T across regularization strengths, with $\lambda = 5e^{-1}$ providing the best AMSE and FWT. The weak performance of Offline-EWC may be due to over-regularization of later tasks, which has been highlighted in theoretical analysis of the approach [54].

As expected, L2T does not achieve AMSE as low as Online-EWC at the same regularization strength ($\lambda = 5e^{-2}$) and performs worse than SGD. This confirms **H3.1**. Additionally, L2T demonstrates poor FWT performance, indicating the L2 penalty is limiting the ability to generalize to unseen data. This suggests that L2 regularization may not be favorable for promoting *knowledge consolidation*. We hypothesized SGD would demonstrate strong ASME but weak BWT, as it shifts the model between task distributions without a CL strategy (Fig. 2.2). Though SGD reaches low AMSE, we also observe low BWT in the range of other methods, and therefore reject **H3.2**. Because SGD provides strong performance without using a CL strategy, this suggests the balanced six task partition used in this scenario is not difficult to learn and remember. If it were a challenging CL task, we would expect catastrophic forgetting to occur and produce a high BWT in the SGD case where no CL strategy is used. Overall, these results support our use of Online-EWC over Offline-EWC and L2T in e.q. 6.1, but the (1) strong performance of SGD and (2) similar performance across tasks indicates that future evaluations may be necessary to understand performance differences.

	MSE E=1	MSE E=5	MSE E=10
FedAvg	0.0327	**0.0270**	**0.0293**
FedProx $\mu = 5e^{-2}$	0.0339	0.0280	0.0312
FedCurv $\lambda = 5e^{-1}$	0.0318	0.0314	0.0300
FedCurv $\lambda = 5e^{-2}$	**0.0316**	0.0334	0.0305
FedCurv $\lambda = 5e^{-3}$	0.0336	0.0314	0.0296
	MSE D=0	MSE D=2	MSE D=4
FedAvg	**0.0298**	**0.0299**	0.0330
FedProx $\mu = 5e^{-2}$	0.0334	0.0323	0.0333
FedCurv $\lambda = 5e^{-1}$	0.0426	0.0370	0.0349
FedCurv $\lambda = 5e^{-2}$	0.0373	0.0360	**0.0326**
FedCurv $\lambda = 5e^{-3}$	0.0354	0.0313	0.0492

Table 6.2: Experiment 4 results showing test MSE under different epochs per round (top, Exp 4A), and dropped clients per round (bottom, Exp 4B). A convergence plot is provided in appendix Fig. A.7.

6.5.2 Experiment 4

Table 6.2 shows results for Experiments 4A and 4B. In Exp. 4A, we observe that methods converge more quickly as E increases. These findings are in line with results reported by FedCurv authors in their comparison (Table 1 of [51]). In Exp. 4B, we observe that convergence stability decreases as D increases. This also replicates findings reported by FedProx authors, who evaluate FL performance in high client dropout settings [40]. FedAvg appears to perform well across many settings of E and D; however, its performance deteriorates in the high client dropout condition with $D = 4$. FedCurv performance is poor across settings, with both FedProx and FedAvg outperforming it in several cases. However, we note that in the $E = 10$ condition, all FedCurv settings improve relative to FedProx. This may be due to the underlying theoretical motivation involving MAP estimates, or random variation in the experiment. However, because FedCurv generally performs poorly across variations of E and D, we reject both **H4.1** and **H4.2**.

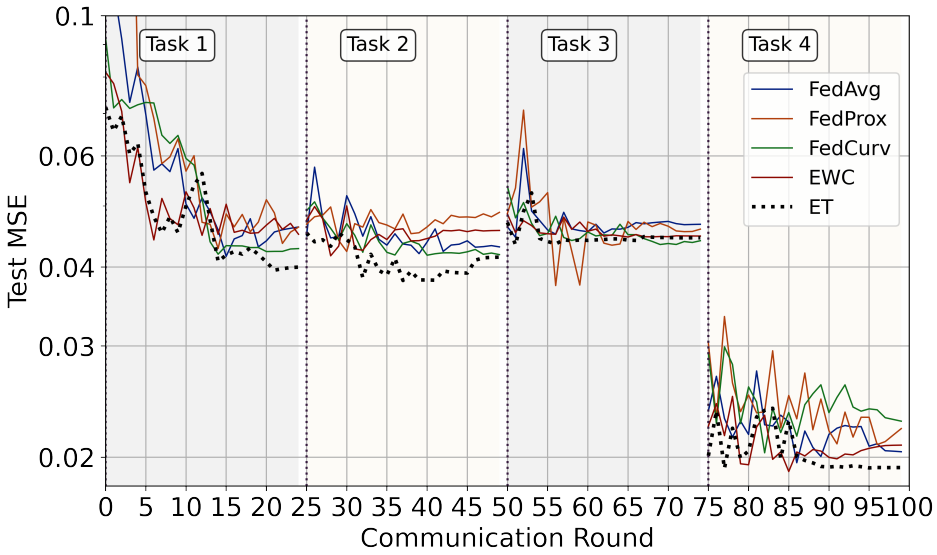

Figure 6.2: Convergence characteristics of EWC, ET, and FedCurv in Experiment 5.

6.5.3 Experiment 5

Fig. 6.2 shows a convergence plot isolating the performance of EWC, FedCurv, and ET. We show three settings where one of λ_1^{EWC}, λ_1^{ET}, and λ_2 are $5e^{-1}$, and others are zero. This plot demonstrates that stability decreases considerably in this scenario with $D = 1$, $E = 5$ compared with the evaluation in Experiment 1B ($D = 0$, $E = 1$). However, Fig. 6.2 shows that penalizing training with the ET term λ_1^{ET} improves stability. This is highlighted at points during training when other methods present MSE oscillations, such as the beginning of training on a new task. The $\lambda_1^{ET} = 5e^{-1}$, $\lambda_1^{EWC} = 0$, $\lambda_2 = 0$ setting visualized here also achieves the best AMSE and FWT across the 29 configurations evaluated. Appendix Fig. A.8 provides a knowledge sharing visualization for this configuration. We see that the ET term contributes to positive performance improvement on $Te^{(1)}$ over training rounds. This is an improvement over Fig. 5.3, which demonstrated a performance plateau after training on $Tr^{(1)}$.

Algorithm			Evaluation Metrics		
			AMSE	**BWT**	**FWT**
FedAvg			0.0391	0.0088	0.0369
FedProx			0.0408	0.0093	0.3711
λ_1^{EWC}	λ_2	λ_1^{ET}			
		$5e^{-1}$	**0.0363**	**0.0083**	**0.0352**
0	0	$5e^{-2}$	0.0374	0.0101	0.0358
		0	0.0388	0.0122	0.0355

Table 6.3: Experiment 5 results in $\lambda_1^{EWC} = 0$, $\lambda_2 = 0$ setting.

We can evaluate Experiment 5 hypotheses by examining how MSE, BWT, and FWT change as a function of λ_1^{ET} when λ_1^{EWC} and λ_2 are not influencing learning ($\lambda_1^{EWC} = 0$ and $\lambda_2 = 0$; Table 6.3). Results show that MSE decreases as λ_1^{ET} increases, confirming **H5.1**. Similarly, because BWT decreases as λ_1^{ET} increases, **H5.2** is also confirmed. There is mixed support for **H5.3**, as $5e^{-1}$ achieves lowest FWT, but this gain is marginal over other regularization strengths. Note that increasing λ_1^{ET} does not improve performance with all configurations of λ_1^{EWC} and λ_2 (Appendix Table A.4). Future work could investigate the interplay between ET, EWC, and FedCurv regularization in greater detail to understand how these three terms interact with one another.

6.6 Analysis

In this chapter, we improve *adaptation quality* and *adaptation time* by facilitating effective *knowledge sharing*. Based on results from Chapter 5, we connect two ideas — EWC and FedCurv — to develop and test a new *Elastic Transfer* regularization approach. The key insight behind this method is that it is possible to decompose FCL training scenarios into a Bayesian formulation, where the posterior consolidates knowledge *across clients and tasks*. During our evaluation, we made several observations. First, naive methods such as SGD and FedAvg performed well, with strong FedAvg performance supporting findings from Chapter 5. Second, L2 regularization approaches (L2T, FedProx) produced higher AMSE than EWC-based methods. Finally, we show that the ET term improves performance stability and results in AMSE and FWT improvements. Further analysis could extend this evaluation to additional scenarios, datasets, and benchmarks to understand how this finding holds in other contexts.

Chapter 7

Conclusion

In this chapter, we summarize the contributions of this work, placing them in the context of long-term HRI challenges. We also discuss limitations and opportunities for future research.

7.1 Summary

This work provides three key contributions, including (1) the formulation of long-term HRI as an FCL problem, (2) an evaluation of the tradeoffs of existing FCL approaches, and (3) a novel regularization-based FCL algorithm. We provide a detailed set of five experiments examining FCL performance as a function of many-to-many and one-to-one personalization scenarios (*Exp. 1*), dataset size (*Exp. 2*), CL-only characteristics (*Exp. 3*), FL-only characteristics (*Exp. 4*), and FCL characteristics under different regularization conditions (*Exp. 5*). We support this evaluation with a theoretical analysis connecting results with existing FL and CL approaches.

One key finding of this work is that *decentralized learning* can achieve similar performance as *centralized learning*. In many-to-many personalization scenarios, we find that FedAvg can achieve AMSE as low as 0.0285, which is in line with the centralized benchmark MSE of 0.0296. Though FL adds communication overhead, it preserves user privacy while enabling adaptation to new users. We identify FL+CL algorithms as providing enhanced adaptation quality and adaptation time compared to STL and CL-only baselines. Through additional analysis, we isolate effective knowledge sharing as the likely mechanism for this performance difference. We also find that architectural strategies such as FedWeIT and APD, which perform well in image recognition benchmarks, do not translate well to long-term HRI scenarios. Therefore, by extending importance-based regularization methods from FL and CL, we provide additional performance improvements through *ET regularization*. Results indicate ET reduces AMSE and FWT compared with FedAvg, FedProx, and EWC/FedCurv.

To summarize how these contributions fit within long-term HRI literature, we return to the challenges discussed in Section 2.1:

- **User Adaptation:** This work provides several improvements for user adaptation in long-term HRI settings. Specifically, we extend the CL-only formulation proposed in [12] to account for generalization *between individuals*. We show how this extensible framework can be applied to many-to-many personalization between groups, and one-to-one personalization to new individuals. This framework may be useful for future personalized FL and long-term HRI research.
- **Data Availability:** We show that effective decentralized learning is possible with datasets that are (1) relatively small, and (2) contain size imbalances between local client-task datasets. Moreover, while a CL-only formulation makes it likely that performance will be poor with initial participants [12], we show that adding FL can accelerate adaptation time through effective knowledge sharing. This may mitigate data availability concerns in future long-term HRI research.
- **Privacy & Data Protection:** Though existing HRI research has considered user privacy [5], to our knowledge, this is the first to examine these in long-term HRI settings. We provide technical mechanisms for improved privacy protection through decentralized training, which may alleviate user privacy concerns.
- **Evaluation Criteria:** We extend the evaluation metrics proposed in the CL-only formulation to capture five holistic dimensions of *adaptation quality*, *adaptation time*, *knowledge sharing*, *privacy*, and *model overhead*. By connecting this broad framework with metrics such as AMSE, BWT, and FWT, we provide concrete tools and evaluation strategies for future long-term HRI research.

This work also advances personalized FL research. Our finding that FedAvg provides robust performance replicates strong FedAvg performance observed during personalized pain estimation [62]. Additionally, [62] found that model averaging reduces performance for some users through *negative transfer*. Because ET regularization improves knowledge sharing by preventing models from overwriting information that is important for other users, this may mitigate negative transfer. Finally, this work is one of the first to examine *temporal adaptivity* in personalized FL [59, 63]. Whereas existing FL methods do not model non-stationary distributions, our FCL formulation proposes Type NI, Type NC, and Type NIC content types for formalizing concept shift over FL training.

7.2 Limitations and Future Work

We highlight the following limitations of this work:

- This research only examines FCL performance using a single dataset and backbone model. Additionally, the FL evaluations reported here are limited in size and scope compared to ones typically reported in the literature [9, 40, 36]. Therefore, it is unknown how well these findings will generalize across related long-term HRI predictive tasks and scenarios.
- A second limitation is that the SocNav1 dataset was collected over a single session with each participant. Therefore, we are not able to examine the ability of FCL methods to capture changing user behaviors over several interactions longitudinally.

In addition to addressing these limitations, there are several other opportunities for future work:

- One opportunity is to use the performance dimensions proposed here to summarize existing personalized FL and longitudinal HRI research. For example, a literature review could examine methods with respect to the five proposed criteria. Because no existing reviews provide this type of comparison to our knowledge, this may be a valuable starting point for the growing field.
- Another avenue of future work is to deploy the proposed framework in a real-world long-term HRI scenario. For example, a group of robots could be deployed as mindfulness coaches and personalize to new users over training sessions. This deployment could validate how well the performance criteria above are satisfied in a realistic scenario.
- A final avenue of future work involves optimizing the ET algorithm for space and communication efficiency. This improvement could follow existing attempts to improve FedCurv communication efficiency [51, 77]. It would also be helpful to benchmark this method in larger-scale FCL training environments with more clients.

In summary, by fostering *personalization* and adding *privacy protections*, approaches such as Federated Continual Learning can bring social robots like Jibo, MIRO, and Pepper closer to everyday life.

Bibliography

[1] Hal Hodson. The first family robot, 2014.

[2] Ben Mitchinson and Tony J Prescott. Miro: a robot "mammal" with a biomimetic brain-based control system. In *Conference on Biomimetic and Biohybrid Systems*, pages 179–191. Springer, 2016.

[3] Amit Kumar Pandey and Rodolphe Gelin. A mass-produced sociable humanoid robot: Pepper: The first machine of its kind. *IEEE Robotics & Automation Magazine*, 25(3):40–48, 2018.

[4] Iolanda Leite, Carlos Martinho, and Ana Paiva. Social robots for long-term interaction: a survey. *International Journal of Social Robotics*, 5(2):291–308, 2013.

[5] Daniel J Butler, Justin Huang, Franziska Roesner, and Maya Cakmak. The privacy-utility tradeoff for remotely teleoperated robots. In *Proceedings of the tenth annual ACM/IEEE international conference on human-robot interaction*, pages 27–34, 2015.

[6] Maggie Astor. Your roomba may be mapping your home, collecting data that could be shared. *The New York Times*, 25, 2017.

[7] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3):50–60, May 2020.

[8] Timothée Lesort, Vincenzo Lomonaco, Andrei Stoian, Davide Maltoni, David Filliat, and Natalia Díaz-Rodríguez. Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58:52–68, 2020.

[9] Jaehong Yoon, Wonyong Jeong, Giwoong Lee, Eunho Yang, and Sung Ju Hwang. Federated continual learning with adaptive parameter communication. In *4th Lifelong Learning Workshop*, 2020.

[10] Luis J Manso, Pedro Nuñez, Luis V Calderita, Diego R Faria, and Pilar Bachiller. Socnav1: A dataset to benchmark and learn social navigation conventions. *Data*, 5(1):7, 2020.

[11] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[12] Nikhil Churamani, Sinan Kalkan, and Hatice Gunes. Continual learning for affective robotics: Why, what and how? *Age*, 30:40, 2020.

[13] Michael A Goodrich and Alan C Schultz. *Human-robot interaction: a survey*. Now Publishers Inc, 2008.

[14] Iroju Olaronke, Ojerinde Oluwaseun, and Ikono Rhoda. State of the art: a study of human-robot interaction in healthcare. *International Journal of Information Engineering and Electronic Business*, 9(3):43, 2017.

[15] Muneeb Ahmad, Omar Mubin, and Joanne Orlando. A systematic review of adaptivity in human-robot interaction. *Multimodal Technologies and Interaction*, 1(3):14, 2017.

[16] Phoebe Liu, Dylan F Glas, Takayuki Kanda, Hiroshi Ishiguro, and Norihiro Hagita. A model for generating socially-appropriate deictic behaviors towards people. *International Journal of Social Robotics*, 9(1):33–49, 2017.

[17] Adriana Tapus et al. Stress game: the role of motivational robotic assistance in reducing user's task stress. *International Journal of Social Robotics*, 7(2):227–240, 2015.

[18] Stanislava Naneva, Marina Sarda Gou, Thomas L Webb, and Tony J Prescott. A systematic review of attitudes, anxiety, acceptance, and trust towards social robots. *International Journal of Social Robotics*, pages 1–23, 2020.

[19] Dag Sverre Syrdal, Kheng Lee Koay, Michael L Walters, and Kerstin Dautenhahn. A personalized robot companion?-the role of individual differences on spatial preferences in hri scenarios. In *RO-MAN 2007-The 16th IEEE International Symposium on Robot and Human Interactive Communication*, pages 1143–1148. IEEE, 2007.

[20] Dana Kulic and Elizabeth Croft. Physiological and subjective responses to articulated robot motion. *Robotica*, 25(1):13, 2007.

[21] Jorge Rios-Martinez, Anne Spalanzani, and Christian Laugier. From proxemics theory to socially-aware navigation: A survey. *International Journal of Social Robotics*, 7(2):137–153, 2015.

[22] SF Chik, CF Yeong, ELM Su, TY Lim, Y Subramaniam, and PJH Chin. A review of social-aware navigation frameworks for service robot in dynamic human environments. *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, 8(11):41–50, 2016.

[23] Luis J Manso, Ronit R Jorvekar, Diego R Faria, Pablo Bustos, and Pilar Bachiller. Graph neural networks for human-aware social navigation. In *Workshop of Physical Agents*, pages 167–179. Springer, 2020.

[24] Jonas Tjomsland, Sinan Kalkan, and Hatice Gunes. Mind your manners! a dataset and a continual learning approach for assessing social appropriateness of robot actions. *arXiv preprint arXiv:2007.12506*, 2020.

[25] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

[26] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT press Cambridge, 2016.

[27] Frank Rosenblatt. Principles of neurodynamics. perceptrons and the theory of brain mechanisms. Technical report, Cornell Aeronautical Lab Inc Buffalo NY, 1961.

[28] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.

[29] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[30] Yong Yu, Xiaosheng Si, Changhua Hu, and Jianxun Zhang. A review of recurrent neural networks: Lstm cells and network architectures. *Neural computation*, 31(7):1235–1270, 2019.

[31] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

[32] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[33] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European semantic web conference*, pages 593–607. Springer, 2018.

[34] Anna Chatzimichali, Ross Harrison, and Dimitrios Chrysostomou. Toward privacy-sensitive human–robot interaction: Privacy terms and human–data interaction in the personal robot era. *Paladyn, Journal of Behavioral Robotics*, 12(1):160–174, 2020.

[35] Google ai blog: Federated learning: Collaborative machine learning without centralized training data. `https://ai.googleblog.com/2017/04/federated-learning-collaborative.html`. (Accessed on 01/14/2021).

[36] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, pages 1273–1282. PMLR, 2017.

[37] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. On the convergence of fedavg on non-iid data. In *International Conference on Learning Representations*, 2020.

[38] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Keith Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *arXiv preprint arXiv:1912.04977*, 2019.

[39] Mehryar Mohri, Gary Sivek, and Ananda Theertha Suresh. Agnostic federated learning. In *International Conference on Machine Learning*, pages 4615–4625. PMLR, 2019.

[40] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *arXiv preprint arXiv:1812.06127*, 2018.

[41] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(2):1–19, 2019.

[42] Li Huang, Andrew L Shea, Huining Qian, Aditya Masurkar, Hao Deng, and Dianbo Liu. Patient clustering improves efficiency of federated machine learning to predict mortality and hospital stay time using distributed electronic medical records. *Journal of biomedical informatics*, 99:103291, 2019.

[43] Yang Zhao, Jun Zhao, Linshan Jiang, Rui Tan, and Dusit Niyato. Mobile edge computing, blockchain and reputation-based crowdsourcing iot federated learning: A secure, decentralized and privacy-preserving system. *arXiv preprint arXiv:1906.10893*, 2019.

[44] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

[45] Gido M Van de Ven and Andreas S Tolias. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.

[46] Davide Maltoni and Vincenzo Lomonaco. Continuous learning in single-incremental-task scenarios. *Neural Networks*, 116:56–73, 2019.

[47] Yu-Xiong Wang, Deva Ramanan, and Martial Hebert. Growing a brain: Fine-tuning by increasing model capacity. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2471–2480, 2017.

[48] Jaehong Yoon, Saehoon Kim, Eunho Yang, and Sung Ju Hwang. Scalable and order-robust continual learning with additive parameter decomposition. *arXiv preprint arXiv:1902.09432*, 2019.

[49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

[50] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.

[51] Neta Shoham, Tomer Avidor, Aviv Keren, Nadav Israel, Daniel Benditkis, Liron Mor-Yosef, and Itai Zeitak. Overcoming forgetting in federated learning on non-iid data. *arXiv preprint arXiv:1910.07796*, 2019.

[52] Robert Hecht-Nielsen. Theory of the backpropagation neural network. In *Neural networks for perception*, pages 65–93. Elsevier, 1992.

[53] Héctor J. Sussmann. Uniqueness of the weights for minimal feedforward nets with a given input-output map. *Neural Networks*, 5(4):589–593, 1992.

[54] Ferenc Huszár. On quadratic penalties in elastic weight consolidation. *arXiv preprint arXiv:1712.03847*, 2017.

[55] David JC MacKay. A practical bayesian framework for backpropagation networks. *Neural computation*, 4(3):448–472, 1992.

[56] Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *International Conference on Machine Learning*, pages 4528–4537. PMLR, 2018.

[57] Sayna Ebrahimi, Mohamed Elhoseiny, Trevor Darrell, and Marcus Rohrbach. Uncertainty-guided continual learning with bayesian neural networks. *arXiv preprint arXiv:1906.02425*, 2019.

[58] Kaidi Cao, Yining Chen, Junwei Lu, Nikos Arechiga, Adrien Gaidon, and Tengyu Ma. Heteroskedastic and imbalanced deep learning with adaptive regularization. *arXiv preprint arXiv:2006.15766*, 2020.

[59] Alysa Ziying Tan, Han Yu, Lizhen Cui, and Qiang Yang. Towards personalized federated learning. *arXiv preprint arXiv:2103.00710*, 2021.

[60] Yuyang Deng, Mohammad Mahdi Kamani, and Mehrdad Mahdavi. Adaptive personalized federated learning. *arXiv preprint arXiv:2003.13461*, 2020.

[61] Yishay Mansour, Mehryar Mohri, Jae Ro, and Ananda Theertha Suresh. Three approaches for personalization with applications to federated learning. *arXiv preprint arXiv:2002.10619*, 2020.

[62] Ognjen Rudovic, Nicolas Tobis, Sebastian Kaltwang, Björn Schuller, Daniel Rueckert, Jeffrey F Cohn, and Rosalind W Picard. Personalized federated deep learning for pain estimation from face images. *arXiv preprint arXiv:2101.04800*, 2021.

[63] Fernando E Casado, Dylan Lema, Roberto Iglesias, Carlos V Regueiro, and Senén Barro. Collaborative and continual learning for classification tasks in a society of devices. *arXiv preprint arXiv:2006.07129*, 2020.

[64] Pablo Barros, German Parisi, and Stefan Wermter. A personalized affective memory model for improving emotion recognition. In *International Conference on Machine Learning*, pages 485–494. PMLR, 2019.

[65] T. Xue, W. Wang, J. Ma, W. Liu, Z. Pan, and M. Han. Progress and prospects of multimodal fusion methods in physical human–robot interaction: A review. *IEEE Sensors Journal*, 20(18):10355–10370, 2020.

[66] *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 2018.

[67] IEEE. Iros 2019 index of papers. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–34, 2019.

[68] *28th IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2019, New Delhi, India, October 14-18, 2019*. IEEE, 2019.

[69] *29th IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2020, Naples, Italy, August 31 - September 4, 2020*. IEEE, 2020.

[70] *14th ACM/IEEE International Conference on Human-Robot Interaction, HRI 2019, Daegu, South Korea, March 11-14, 2019*. IEEE, 2019.

[71] Tony Belpaeme, James Young, Hatice Gunes, and Laurel D. Riek, editors. *Companion of the 2020 ACM/IEEE International Conference on Human-Robot Interaction, HRI 2020, Cambridge, UK, March 23-26, 2020*. ACM, 2020.

[72] *14th IEEE International Conference on Automatic Face & Gesture Recognition, FG 2019, Lille, France, May 14-18, 2019*. IEEE, 2019.

[73] *15th IEEE International Conference on Automatic Face and Gesture Recognition, FG 2020, Buenos Aires, Argentina, November 16-20, 2020*. IEEE, 2020.

[74] Jacob Cohen. Weighted kappa: nominal scale agreement provision for scaled disagreement or partial credit. *Psychological bulletin*, 70(4):213, 1968.

[75] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, and Eftychios Protopapadakis. Deep learning for computer vision: A brief review. *Computational intelligence and neuroscience*, 2018, 2018.

[76] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He,

George Karypis, Jinyang Li, and Zheng Zhang. Deep graph library: A graph-centric, highly-performant package for graph neural networks, 2020.

[77] Xin Yao and Lifeng Sun. Continual local training for better initialization of federated models. In *2020 IEEE International Conference on Image Processing (ICIP)*, pages 1736–1740. IEEE, 2020.

Appendix A

Supporting Content

Algorithm 2: *Federated Averaging (FedAvg)*

Input: Client partitions $\{\mathcal{D}_c\}$, Rounds of training R, Epochs of training per round E, Set of clients C, Mini-batch size B

Output: Optimized global model θ_G^*

Initialize global model θ_G
for *round* $r \leftarrow 1$ ***to*** R **do**
    Select available clients $C_r \subset C$
    for *client* $c \in C_r$ **do**
        $\theta_c^{r+1} \leftarrow ClientUpdate(\theta_G^r)$ ▷ `Train on client` c.
    end
    $\theta_G^{r+1} \leftarrow \sum_{c \in C_r} \frac{n_c}{n} \theta_c^{r+1}$ ▷ `Compute average update.`
end
return θ_G^r

function *ClientUpdate*(θ_G^r)
    $\mathcal{B} \leftarrow$ split $\mathcal{D}_c$ into batches of size B
    $\theta \leftarrow \theta_G^r$ ▷ `Update local parameters.`
    for *epoch* $e \leftarrow 1$ ***to*** E **do**
        for *batch* $b \in \mathcal{B}$ **do**
            $\theta \leftarrow \theta - \eta \Delta \ell(b; \theta)$ ▷ `Mini-batch SGD update.`
        end
    end
    return θ to server

Algorithm 3: Federated Weighted Inter-client Transfer

Input: Client-task datasets $\{\mathcal{D}_c^{(1:T)}\}_{c\in C, t\in\{1,\dots,T\}}$, epochs-per-round E, rounds-per-task R, number of clients C, number of tasks T

Output: Optimized global model θ_G

Initialize global model θ_G and distribute to clients
for $t \leftarrow 1$ ***to*** T **do**
  for $r \leftarrow 1$ ***to*** R **do**
    Select available clients $C_r \subset C$
    for $c \in C_r$ **do**
      Distribute θ_G^r and $\{A_j^{(t-1,R)}\}_{j\in C}$ to each client c
      Minimize local loss (Eq. 2 in [9]) for each local CL problem
      Compute sparse parameter $\theta_c^{(t,r)} = B_c^{(t)} \circ m_c^{(t)}$
      Transmit $\theta_c^{(t,r)}$ and $A_c^{(t-1,R)}$ to server
      Compute $\theta_G^{(r+1)} \leftarrow \frac{1}{|C|}\sum_{c\in C}\theta_c^{(t,r)}$
return θ_G

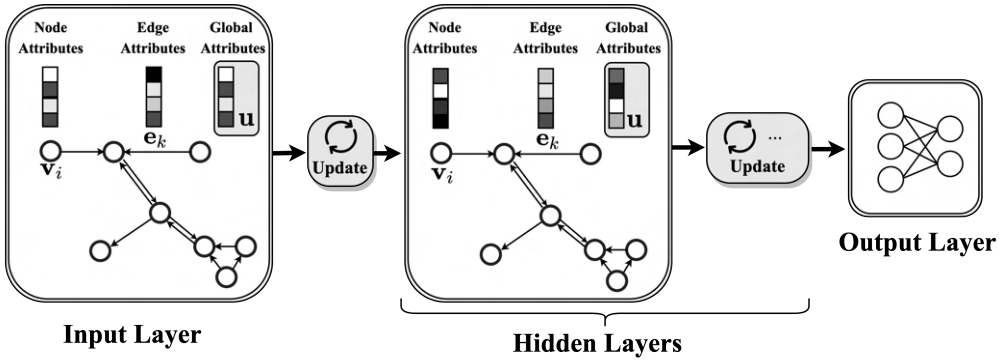

Figure A.1: Layers in a GCN update node ($\mathbf{v}_i$), edge ($\mathbf{e}_k$), and global ($\mathbf{u}$) attributes defined over a fixed graph topology and connect to an output layer to perform the task at hand.

Hyperparameter	**Settings Sampled**
Network Layers	2, 3, 4, 5, 6
Hidden Layer Size	32, 64, 96, 128, 256, 350, 512
Weight Decay (γ)	0.0001, 0.001, 0.005
Batch Size	175, 256, 350, 450, 512, 800, 1600

Table A.1: Hyperparameter settings sampled during the GNN validation experiment with the DGL+GCN architecture.

Run	Train MSE	Train R2	Val. MSE	Val. R2	Test MSE	Test R2
Rand. 1	0.0745	0.3702	0.0733	0.3873	0.0800	0.3109
Rand. 2	0.0542	0.5403	0.0549	0.5408	0.0592	0.4898
Rand. 3	0.0968	0.1813	0.0955	0.2019	0.1030	0.1127
Rand. 4	0.0837	0.2922	0.0871	0.2719	0.0850	0.2679
Rand. 5	0.1118	0.0548	0.1097	0.0834	0.1193	-0.0278
Rand. 6	0.0573	0.5143	0.0583	0.5125	0.0628	0.4589
Rand. 7	0.0632	0.4654	0.0685	0.4276	0.0732	0.3692
Rand. 8	0.0585	0.5062	0.0598	0.5004	0.0644	0.4448
Rand. 9	0.3478	-1.9383	0.3750	-2.1320	0.3639	-2.1342
Rand. 10	0.1301	-0.1007	0.1292	-0.0792	0.1365	-0.1758
Fine-Tuned	**0.0289**	**0.7553**	**0.0287**	**0.7600**	**0.0296**	**0.7447**

Table A.2: Results of GNN hyper-parameter search conducted with the DGL+GCN architecture. The fine-tuned configuration has 8 layers, 32 hidden units per layer, and was trained with $\gamma = 1e^{-4}$, and a mini-batch size of 256 examples.

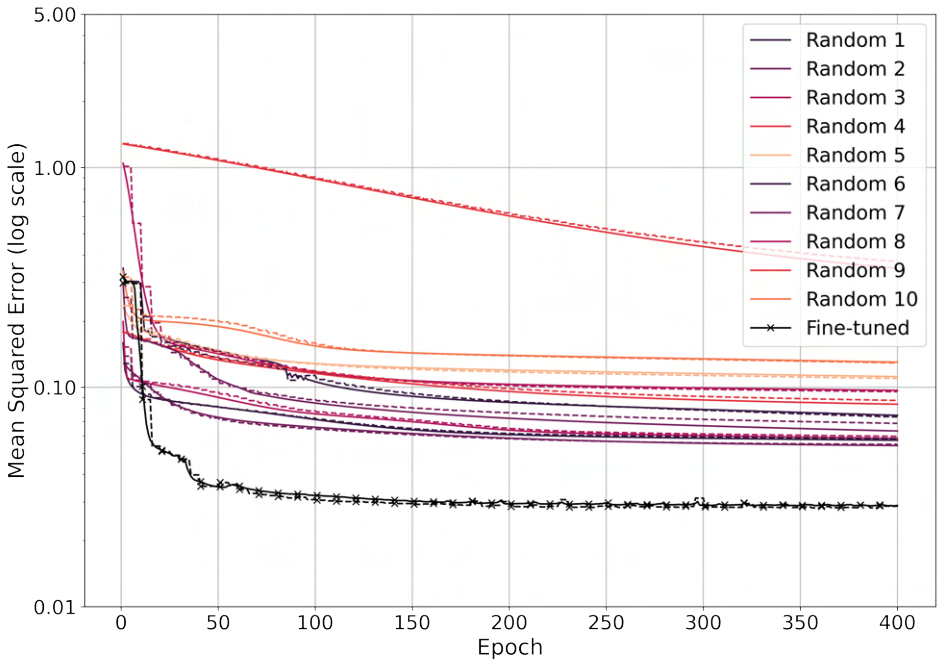

Figure A.2: MSE on $\mathcal{D}_{train}$ (solid) and $\mathcal{D}_{valid}$ (dotted) MSE during the centralized learning benchmark experiment.

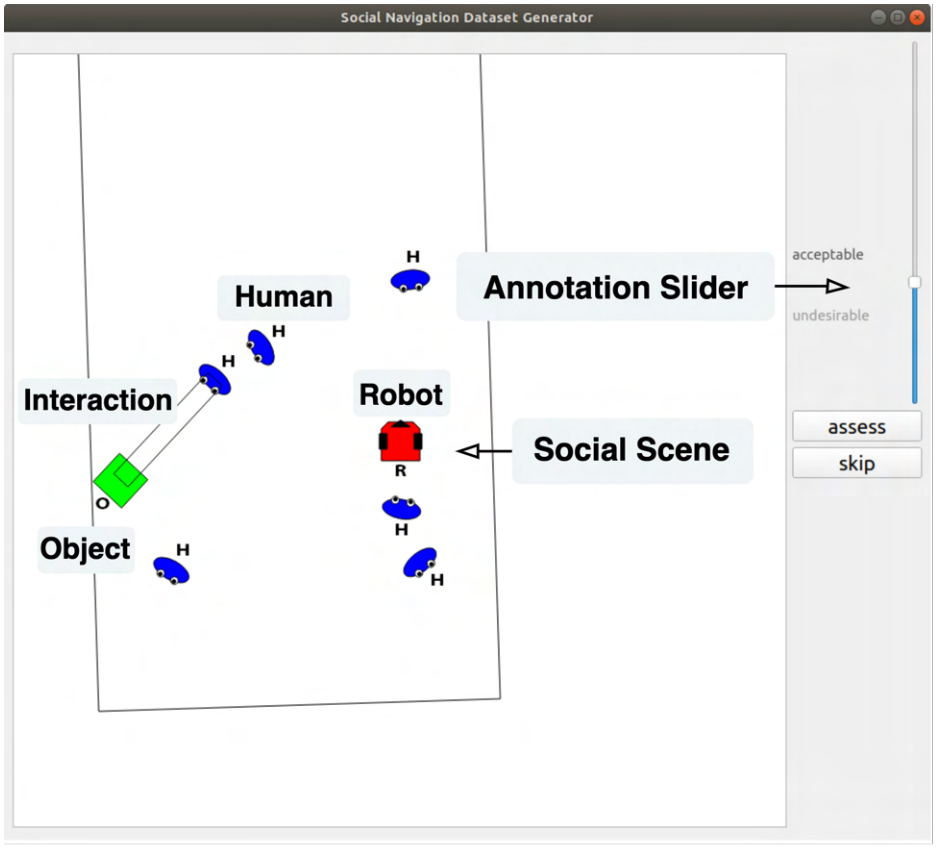

Figure A.3: The SocNav1 annotation interface. This image includes the original screenshot presented in [10] supplemented by labels for humans (blue), robots (red), objects (green), and interactions (parallel lines).

	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6
Client 1	1770, 190, 190	1034, 111, 111	4230, 453, 454	928, 99, 100	3722, 399, 399	1304, 140, 140

Table A.3: Client-Task dataset sizes for Experiment 3. The same client-task datasets are used in a transposed client-task layout for Experiment 4.

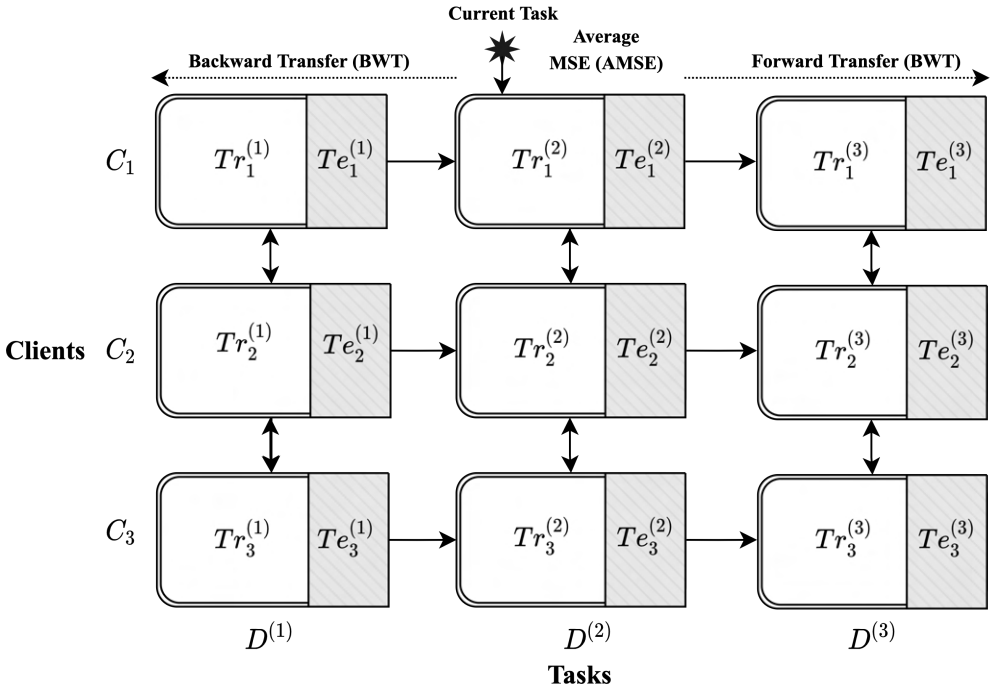

Figure A.4: Schematic of training partitions $Tr_c^{(t)}$ and test partitions $Te_c^{(t)}$ across clients and tasks in an FCL formulation. At this time, the algorithm is training on Task 2 and has access to $Tr_c^{(2)}$ (entries in row $Tr^{(2)}$ of Fig. 4.3). Scores on $Te_c^{(1)}$, $Te_c^{(2)}$, and $Te_c^{(3)}$ contribute to BWT, AMSE, and FWT metrics, respectively.

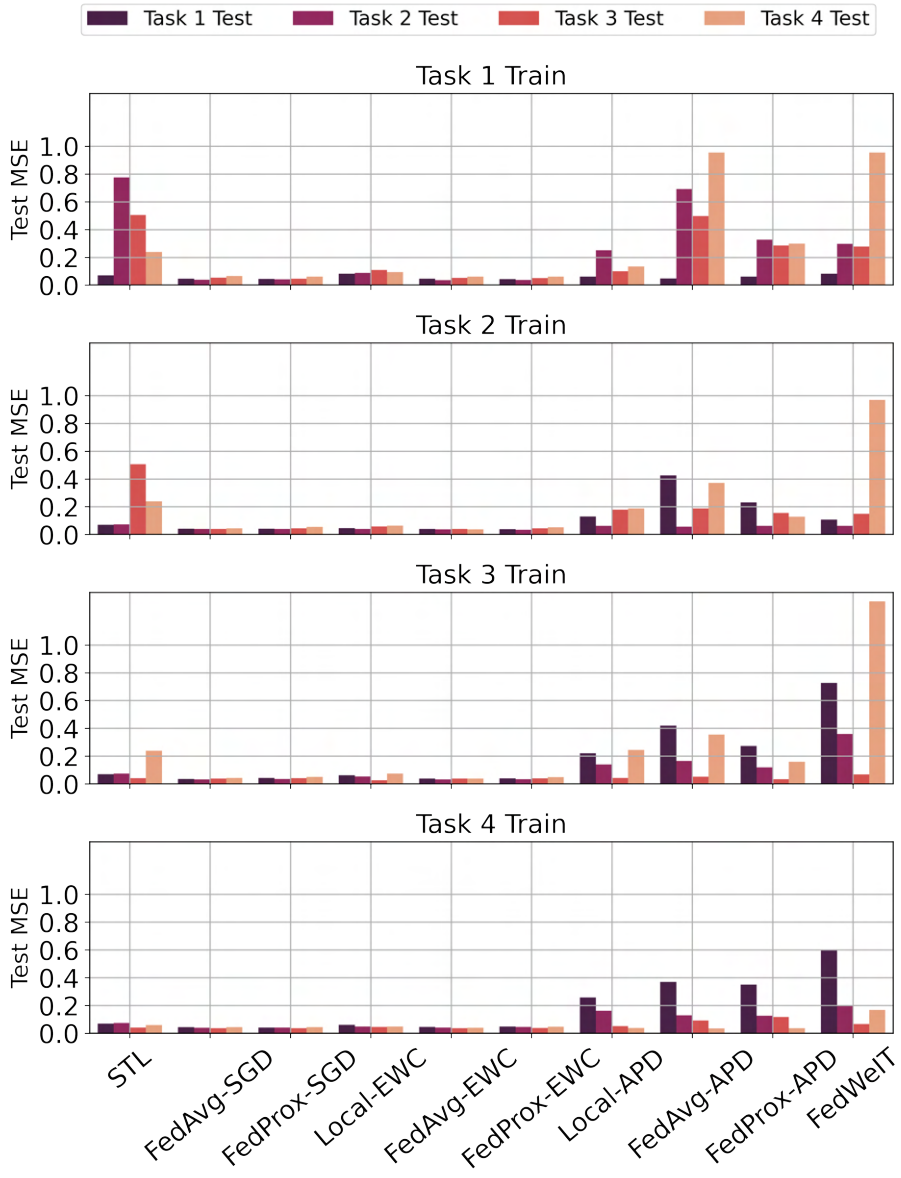

Figure A.5: MSE on $Te_c^{(t)}$ over training tasks. Visualization follows layout P, where $Tr_c^{(t)}$ occupies rows and $Te_c^{(t)}$ occupies columns.

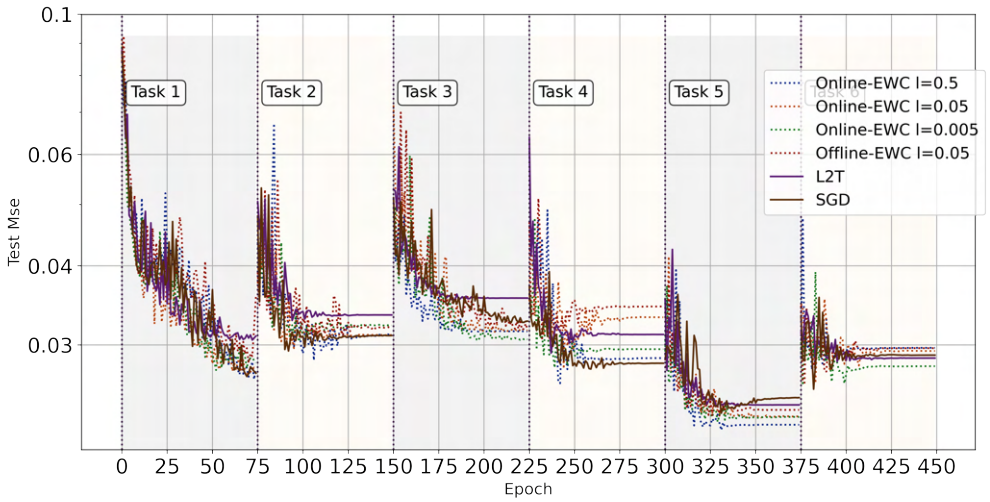

Figure A.6: Experiment 3 Test MSE convergence over tasks.

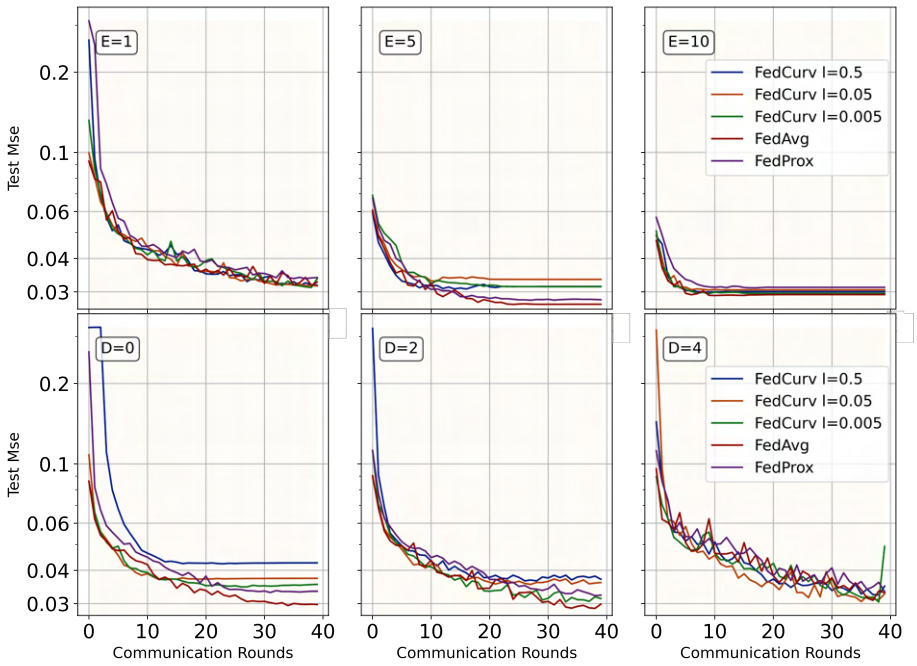

Figure A.7: Experiment 4 results showing convergence under different epochs per round (E, Exp 4.A), and dropped clients per round (D, Exp 4.B).

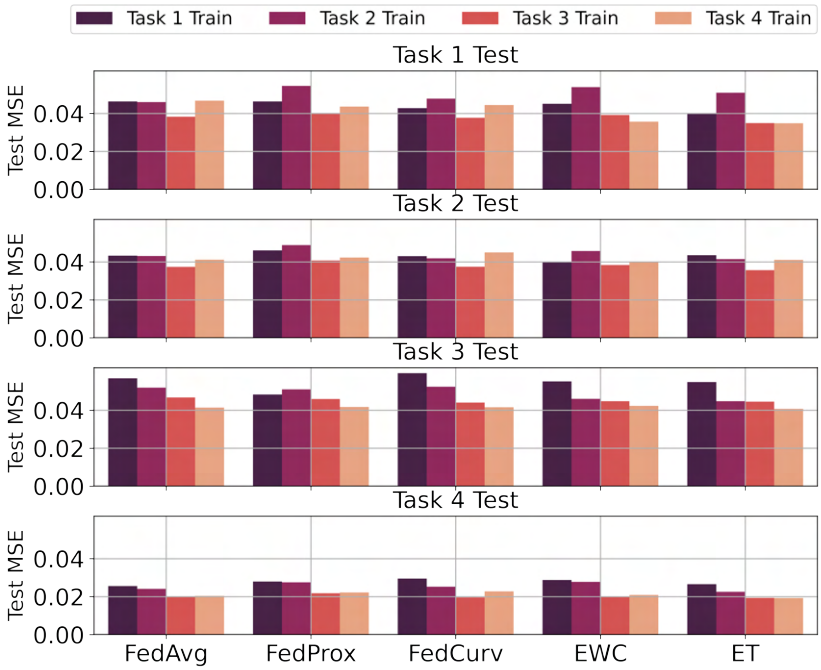

Figure A.8: Test MSE on tasks over training rounds. FedCurv, EWC, and ET baselines demonstrate settings where the respective terms λ_1^{EWC}, λ_1^{ET}, and λ_2 are $\lambda = 5e^{-1}$ and others are zero.

Algorithm			Evaluation Metrics		
			AMSE	**BWT**	**FWT**
FedAvg			0.0391	0.0088	0.0369
FedProx			0.0408	0.0093	0.3711
λ_1^{EWC}	λ_2	λ_1^{ET}			
$5e^{-1}$	$5e^{-1}$	$5e^{-1}$	0.0408	0.0120	0.0380
		$5e^{-2}$	0.0392	0.0114	0.0391
		0	0.0389	0.0103	0.0395
	$5e^{-2}$	$5e^{-1}$	0.0389	0.0108	0.0407
		$5e^{-2}$	0.0385	0.0115	0.0390
		0	0.0397	0.0113	0.0450
	0	$5e^{-1}$	0.0383	0.0107	0.0383
		$5e^{-2}$	0.0369	0.0124	0.0355
		0	0.0391	0.0085	0.0362
$5e^{-2}$	$5e^{-1}$	$5e^{-1}$	0.0396	**0.0062**	0.0400
		$5e^{-2}$	0.0400	0.0111	0.0395
		0	0.0384	0.0115	0.0390
	$5e^{-2}$	$5e^{-1}$	0.0402	0.0115	0.0381
		$5e^{-2}$	0.0390	0.0080	0.0392
		0	0.0391	0.0086	0.0382
	0	$5e^{-1}$	0.0376	0.0107	0.0375
		$5e^{-2}$	0.0390	0.0091	0.0391
		0	0.0386	0.0128	0.0373
0	$5e^{-1}$	$5e^{-1}$	0.0403	0.0112	0.0412
		$5e^{-2}$	0.0392	0.0118	0.0380
		0	0.0378	0.0092	0.0382
	$5e^{-2}$	$5e^{-1}$	0.0388	0.0122	0.0396
		$5e^{-2}$	0.0374	0.0108	0.0384
		0	0.0404	0.0137	0.0411
	0	$5e^{-1}$	**0.0363**	0.0083	**0.0352**
		$5e^{-2}$	0.0374	0.0101	0.0358
		0	0.0388	0.0122	0.0355

Table A.4: Experiment 5 results comparing the strength of λ_1^{EWC}, λ_1^{ET}, and λ_2 with FedAvg and FedProx.

Appendix B

CC-BY License Image Credits

Park by Creaticca Creative Agency from the Noun Project

Home by Nawicon from the Noun Project

Museum by Wolf Böse from the Noun Project

Woman by Lluisa Iborra from the Noun Project

Man by RROOK from the Noun Project

Old lady by parkjisun from the Noun Project

Commercial house building by Vectors Point from the Noun Project

University of queensland by Vectors Point from the Noun Project

Woman by Lluisa Iborra from the Noun Project

Businessman by Lluisa Iborra from the Noun Project

Businessman by Lluisa Iborra from the Noun Project

Robot by Andrejs Kirma from the Noun Project

Person by ProSymbols from the Noun Project

Mother by ProSymbols from the Noun Project

Female Anchor by ProSymbols from the Noun Project

Student by ProSymbols from the Noun Project